



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



**TFG del Grado en Ingeniería
Informática**

**Evolución de un editor
Entidad-Relación con React y
mxGraph
Documentación Técnica**



Presentado por Steven Paul Alba Alba
en Universidad de Burgos — 28 de junio
de 2026

Tutor: Jesús Manuel Maudes Raedo

Índice general

Índice general	i
Índice de figuras	iii
Índice de tablas	v
Apéndice A Plan de Proyecto Software	1
A.1. Introducción	1
A.2. Planificación temporal	1
A.3. Estudio de viabilidad	7
Apéndice B Especificación de Requisitos	13
B.1. Introducción	13
B.2. Objetivos generales	13
B.3. Catálogo de requisitos	14
B.4. Especificación de requisitos	16
Apéndice C Especificación de diseño	33
C.1. Introducción	33
C.2. Diseño de datos	33
C.3. Diseño arquitectónico	38
C.4. Diseño procedimental	42
C.5. Diseño de interfaces	47
Apéndice D Documentación técnica de programación	49
D.1. Introducción	49
D.2. Estructura de directorios	50

D.3. Manual del programador	54
D.4. Organización interna del código fuente	58
D.5. Pruebas del sistema	63
Apéndice E Documentación de usuario	71
E.1. Introducción	71
E.2. Requisitos de usuario	72
E.3. Instalación	72
E.4. Manual del usuario	73
Apéndice F Anexo de sostenibilización curricular	91
F.1. Introducción	91
F.2. Competencias de sostenibilidad adquiridas	91
F.3. Reflexión final	93
Bibliografía	95

Índice de figuras

A.1. Milestone 1: preparación del entorno y comprensión del proyecto.	3
A.2. Milestone 2: estabilización del núcleo y entidades débiles.	4
A.3. Milestone 3: refactorización, optimización y atributos avanzados.	4
A.4. Milestone 4: implementación de relaciones ternarias.	5
A.5. Milestone 5: soporte inicial de ISA y mejoras finales de usabilidad.	5
A.6. Cronograma resumido de los sprints adaptados del proyecto. . .	6
B.1. Diagrama de casos de uso de UBU E-R App.	18
C.1. Metamodelo E/R de la representación interna de un diagrama. .	34
C.2. Correspondencia simplificada entre el metamodelo, la representa- ción JSON y los procesos de la aplicación.	36
C.3. Diagrama UML de despliegue de la aplicación.	39
C.4. Diagrama UML de componentes de la aplicación.	40
C.5. Flujo de validación, transformación relacional y generación de SQL.	45
C.6. Diagrama de secuencia de validación, transformación relacional y generación de SQL.	46
D.1. Ejecución de las pruebas unitarias del proyecto mediante Vitest.	65
D.2. Ejecución de las pruebas de extremo a extremo mediante Playw- right.	66
D.3. Comprobación de análisis estático y construcción de producción del proyecto.	70
E.1. Pantalla principal de UBU E-R App con las entidades iniciales del ejemplo.	74
E.2. Panel lateral con herramientas de creación y acciones contextuales de una entidad seleccionada.	75

E.3. Panel lateral con acciones generales, importación, exportación y ayuda del editor.	76
E.4. Entidades <i>Empleados</i> y <i>Oficinas</i> con sus atributos y claves primarias.	78
E.5. Configuración de los participantes de la relación <i>Trabaja_en</i> . . .	79
E.6. Configuración de cardinalidades de la relación <i>Trabaja_en</i>	80
E.7. Mensaje de validación asociado a la entidad <i>Empleados</i>	81
E.8. Script SQL compatible con PostgreSQL generado a partir del ejemplo.	82
E.9. Importación de un diagrama desde un fichero JSON.	83
E.10. Selección múltiple de elementos en el lienzo.	84
E.11. Generación de estructuras predefinidas en el editor.	86
E.12. Información general de UBU E-R App.	89

Índice de tablas

A.1. Estimación de horas dedicadas por bloque de trabajo.	8
A.2. Coste total estimado del proyecto.	9
A.3. Licencias principales consideradas en el proyecto.	10
B.1. CU-1 Crear y editar entidades.	19
B.2. CU-2 Crear y editar atributos.	20
B.3. CU-3 Configurar atributos especiales.	21
B.4. CU-4 Crear y configurar relaciones binarias.	22
B.5. CU-5 Crear y configurar relaciones ternarias y roles.	23
B.6. CU-6 Modelar entidades débiles y relaciones identificadoras. . .	24
B.7. CU-7 Modelar jerarquías ISA sencillas.	25
B.8. CU-8 Validar explícitamente el diagrama.	26
B.9. CU-9 Generar SQL a partir del diagrama.	27
B.10. CU-10 Guardar y recuperar el diagrama localmente.	28
B.11. CU-11 Exportar e importar diagramas en JSON.	29
B.12. CU-12 Generar estructuras básicas de ejemplo.	30
B.13. CU-13 Exportar la representación visual del diagrama.	31
B.14. CU-14 Utilizar acciones de apoyo de la interfaz.	32
C.1. Diccionario de datos simplificado del metamodelo del diagrama.	37

Apéndice A

Plan de Proyecto Software

A.1. Introducción

En este anexo se describe la planificación seguida durante el desarrollo del Trabajo Fin de Grado, así como el estudio de viabilidad asociado. A diferencia de un proyecto iniciado desde cero, este trabajo parte de una aplicación heredada, por lo que la planificación no se centró únicamente en añadir nuevas funcionalidades, sino también en comprender el sistema existente, estabilizar su comportamiento y ampliar sus capacidades de forma controlada.

La planificación temporal recoge la forma en la que se organizó el trabajo, las fases principales del desarrollo y los mecanismos utilizados para realizar el seguimiento del proyecto. En este proceso se utilizaron issues y milestones de GitHub, ramas de desarrollo, revisiones con el tutor y despliegues funcionales en Vercel, que permitieron comprobar el estado de la aplicación desde el navegador sin depender de una instalación local.

El estudio de viabilidad analiza los costes que tendría un proyecto de estas características en un contexto profesional, así como las consideraciones legales relacionadas con las licencias de las herramientas y librerías utilizadas.

A.2. Planificación temporal

Metodología y organización del trabajo

El desarrollo del proyecto se organizó mediante una metodología ágil basada en Scrum, adaptada al contexto de un Trabajo Fin de Grado indivi-

dual. No se aplicó Scrum de forma completa, ya que no existía un equipo de desarrollo con roles diferenciados ni reuniones diarias, pero sí se utilizaron varios de sus principios: división del trabajo en bloques, definición de tareas concretas, revisión periódica de resultados y adaptación de la planificación según el avance del proyecto.

La organización práctica del trabajo se apoyó principalmente en GitHub. Los issues se utilizaron para registrar tareas concretas de análisis, implementación, pruebas, revisión y documentación. Estos issues actuaron como backlog del proyecto y permitieron mantener trazabilidad entre los objetivos planteados y el trabajo realizado.

Para facilitar su clasificación, se definió un conjunto de labels en GitHub. Estas etiquetas permitieron distinguir la naturaleza de cada tarea, por ejemplo si estaba relacionada con la configuración del entorno, el despliegue, la documentación, las pruebas, el modelo Entidad-Relación, mejoras funcionales, cuestiones teóricas o elementos centrales del TFG. Entre las etiquetas utilizadas se encontraban *setup*, *deployment*, *documentation*, *testing*, *er-model*, *enhancement*, *theory* y *tfg-core*.

Los milestones agruparon issues relacionados y se utilizaron como sprints adaptados. Cada milestone representó un bloque de trabajo de mayor duración que un sprint Scrum convencional, centrado en una parte concreta de la evolución del editor. Esta adaptación resultó adecuada para el proyecto, ya que se trataba de un TFG individual basado en una aplicación heredada y no de un desarrollo en equipo desde cero.

Además, se trabajó con ramas de desarrollo para separar cambios funcionales o de documentación. Los despliegues en Vercel permitieron publicar versiones funcionales de la aplicación y facilitaron su revisión desde el navegador, sin necesidad de instalar el proyecto en local.

En este contexto, la equivalencia entre Scrum y la organización real del TFG puede resumirse de la siguiente forma: los issues actuaron como tareas del backlog, las labels permitieron clasificar el trabajo, los milestones funcionaron como sprints adaptados, las versiones integradas y desplegadas como incrementos revisables, y las revisiones con el tutor como mecanismo de seguimiento y ajuste del trabajo.

Sprints adaptados del proyecto

El desarrollo se dividió en varios sprints adaptados, correspondientes a los milestones definidos en GitHub. A diferencia de una planificación Scrum

estricta, estos sprints no tuvieron una duración homogénea, sino que se ajustaron a la complejidad de cada bloque de trabajo y al avance real del proyecto.

Cada sprint adaptado agrupó issues relacionados con un objetivo común. Estos issues no se limitaron a la implementación de una funcionalidad aislada, sino que también incluyeron tareas de análisis, revisión del comportamiento, validación, pruebas, documentación y ajustes derivados de las revisiones realizadas.

El criterio seguido en todos los sprints fue mantener la coherencia entre el lienzo del editor, el modelo interno, las validaciones, la transformación al modelo relacional y la generación SQL. Por este motivo, estas comprobaciones se consideran transversales a cada milestone y no fases independientes del proyecto.

En las figuras incluidas a continuación se muestran los milestones correspondientes en GitHub, donde se recoge su descripción, estado de avance, fecha prevista e issues asociados.

Sprint adaptado 1: preparación del entorno y comprensión del proyecto

Este sprint se corresponde con el milestone *Environment setup and project understanding*. Su objetivo fue preparar el entorno de desarrollo y comprender el estado inicial de la aplicación heredada antes de introducir cambios funcionales relevantes. Los issues asociados pueden consultarse en el [milestone correspondiente en GitHub](#).

Durante esta fase se revisó la estructura del repositorio, se configuró la ejecución local, se comprobaron los comandos principales del proyecto y se realizó un primer despliegue en Vercel. El resultado fue disponer de una base de trabajo preparada y una organización inicial mediante issues, labels, milestones y ramas de desarrollo.

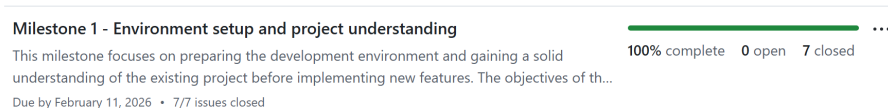


Figura A.1: Milestone 1: preparación del entorno y comprensión del proyecto.

Sprint adaptado 2: estabilización del núcleo y entidades débiles

Este sprint se corresponde con el milestone *Core stabilization and Weak Entities implementation*. Su objetivo fue estabilizar partes relevantes del núcleo del editor y comenzar la ampliación del modelo Entidad-Relación mediante el soporte de entidades débiles. Los issues asociados pueden consultarse en el [milestone correspondiente en GitHub](#).

El trabajo realizado permitió revisar la sincronización entre la representación visual y el modelo interno, así como integrar entidades débiles en el flujo de la aplicación. Esta ampliación se comprobó teniendo en cuenta su efecto sobre el lienzo, el modelo interno, las validaciones, la transformación relacional y la generación SQL.

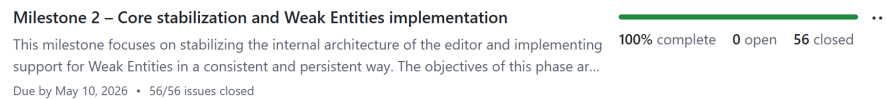


Figura A.2: Milestone 2: estabilización del núcleo y entidades débiles.

Sprint adaptado 3: refactorización, optimización y atributos avanzados

Este sprint se corresponde con el milestone *Code refactoring, optimization and advanced attributes implementation*. Su objetivo fue reducir la complejidad de algunas partes del sistema y ampliar el soporte de atributos del modelo Entidad-Relación. Los issues asociados pueden consultarse en el [milestone correspondiente en GitHub](#).

Durante este sprint se trabajó en la mejora de la estructura interna del editor y en el soporte de atributos avanzados, como atributos compuestos y multivaluados. Estos cambios se revisaron considerando su representación visual, almacenamiento interno, validaciones, transformación relacional y generación SQL.

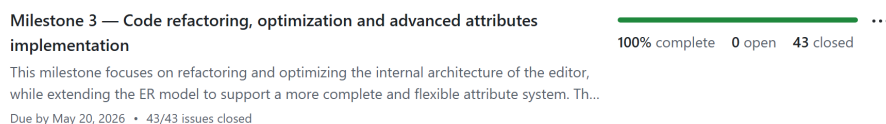


Figura A.3: Milestone 3: refactorización, optimización y atributos avanzados.

Sprint adaptado 4: implementación de relaciones ternarias

Este sprint se corresponde con el milestone *Ternary relationships implementation*. Su objetivo fue ampliar el editor para soportar relaciones ternarias, manteniendo el comportamiento existente de los elementos ya implementados. Los issues asociados pueden consultarse en el [milestone correspondiente en GitHub](#).

La implementación de relaciones ternarias afectó a varias partes del sistema: representación en el lienzo, participantes de la relación, roles, atributos propios, modelo interno, transformación relacional y SQL generado. El resultado fue la incorporación controlada de relaciones ternarias dentro del alcance del TFG.

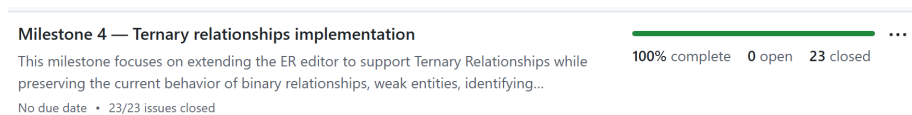


Figura A.4: Milestone 4: implementación de relaciones ternarias.

Sprint adaptado 5: soporte inicial de ISA y mejoras finales de usabilidad

Este sprint se corresponde con el milestone *ISA initial support and editor usability refinements*. Su objetivo fue introducir un soporte inicial para jerarquías ISA y cerrar mejoras de usabilidad e interacción solicitadas o detectadas durante la revisión del editor. Los issues asociados pueden consultarse en el [milestone correspondiente en GitHub](#).

El soporte de ISA se abordó de forma limitada y controlada, incluyendo generalización, especializaciones, clave heredada, transformación al modelo relacional y generación SQL asociada. Además, se incorporaron mejoras finales como ayuda integrada, información del proyecto, selección múltiple visible, operaciones sobre multiselección, exportación visual, ajuste de vista y opciones de importación. El resultado fue una versión más completa y usable de UBU E-R App, manteniendo explícitas las limitaciones de alcance.

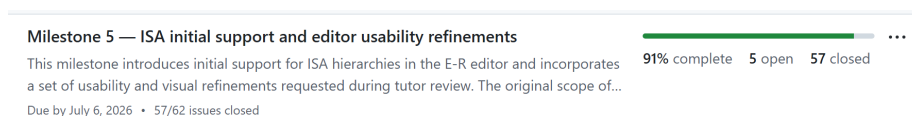


Figura A.5: Milestone 5: soporte inicial de ISA y mejoras finales de usabilidad.

Cronograma general

La Figura A.6 muestra una visión temporal resumida de los sprints adaptados del proyecto. Su finalidad no es representar actividades aisladas, sino situar en el tiempo los principales bloques funcionales abordados durante el desarrollo.

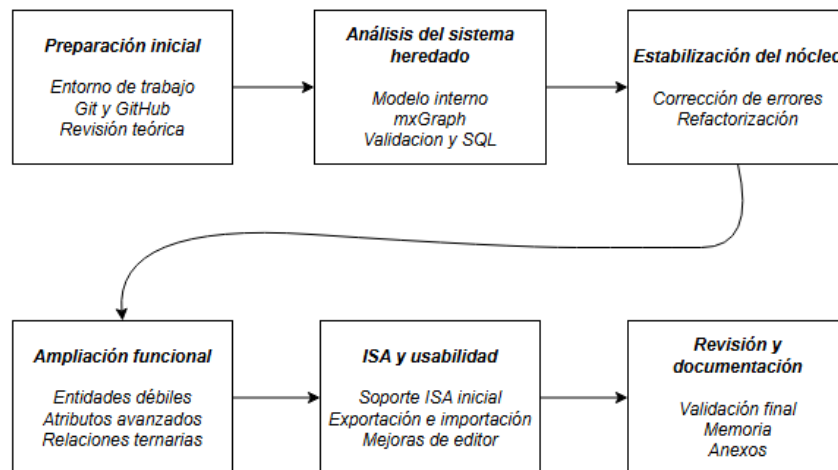


Figura A.6: Cronograma resumido de los sprints adaptados del proyecto.

Seguimiento y validación del proyecto

El seguimiento del proyecto se realizó a partir de los milestones e issues definidos en GitHub, junto con las revisiones mantenidas con el tutor. Estas revisiones permitieron comprobar el avance del trabajo, ajustar prioridades y detectar problemas de enfoque tanto en la aplicación como en la memoria y los anexos.

Los despliegues en Vercel fueron un apoyo importante en este proceso, ya que permitieron revisar versiones funcionales del editor directamente desde el navegador, sin necesidad de instalar el proyecto en local. De esta forma, los cambios integrados podían comprobarse sobre una versión accesible de la aplicación.

La validación técnica se realizó de forma progresiva durante el desarrollo. Algunas partes del proyecto se comprobaron mediante pruebas automáticas, especialmente aquellas relacionadas con validación, transformación relacional, generación SQL y comportamientos concretos del editor. Otras partes

requirieron revisión manual, sobre todo las relacionadas con la interacción visual en el lienzo, la sincronización entre la vista y el modelo interno, la importación y exportación de diagramas y el comportamiento general de la interfaz.

Esta validación no se concentró únicamente al final del proyecto. En cada sprint adaptado se revisó que las funcionalidades desarrolladas mantuvieran la coherencia entre el lienzo, el modelo interno, las validaciones, la transformación al modelo relacional y la generación SQL.

A.3. Estudio de viabilidad

El estudio de viabilidad permite valorar si el proyecto puede llevarse a cabo con los recursos disponibles y dentro del contexto académico en el que se desarrolla. En este caso, se analiza principalmente la viabilidad económica y la viabilidad legal.

Viabilidad económica

La viabilidad económica del proyecto puede analizarse desde dos puntos de vista. Por un lado, el coste directo real para el desarrollo del TFG fue reducido, ya que se utilizó un equipo propio y herramientas gratuitas o de acceso libre. Por otro lado, para valorar el esfuerzo necesario en un contexto profesional, se realiza una estimación económica del trabajo desarrollado.

El principal coste del proyecto corresponde al tiempo dedicado al análisis de la aplicación heredada, la implementación de mejoras, la revisión funcional, las pruebas y la elaboración de la documentación. Al tratarse de un proyecto basado en una aplicación existente, una parte importante del esfuerzo se destinó a comprender el sistema previo y a garantizar que las ampliaciones realizadas mantuvieran la coherencia entre el lienzo, el modelo interno, las validaciones, la transformación relacional y la generación SQL.

Costes de personal

Para estimar el coste de personal se ha tomado como referencia una dedicación total aproximada de 713 horas. Esta estimación incluye las tareas de preparación inicial, análisis del sistema heredado, implementación, validación, pruebas, revisión manual y documentación final.

La Tabla A.1 muestra una distribución aproximada de horas por bloques de trabajo.

Bloque de trabajo	Horas estimadas
Preparación del entorno y comprensión inicial del proyecto	65 h
Estabilización del núcleo y soporte de entidades débiles	205 h
Refactorización, optimización y atributos avanzados	145 h
Implementación de relaciones ternarias	115 h
Soporte inicial de ISA y mejoras finales de usabilidad	120 h
Memoria, anexos y revisión final	63 h
Total	713 h

Tabla A.1: Estimación de horas dedicadas por bloque de trabajo.

Para realizar la estimación económica se ha utilizado un coste orientativo de 15 €/h, correspondiente a una valoración prudente del trabajo de un perfil junior en un contexto académico. Con este criterio, el coste de personal estimado es el siguiente:

$$713 \text{ h} \times 15 \text{ €/h} = 10,695 \text{ €} \quad (\text{A.1})$$

Por tanto, el coste estimado de personal asciende a 10.695 €.

Costes de hardware

El desarrollo se realizó utilizando un ordenador de sobremesa propio, valorado aproximadamente en 1.200 €. Aunque no supuso un gasto directo específico para este TFG, se puede imputar una parte de su coste mediante amortización.

Para esta estimación se considera una vida útil de cuatro años, equivalente a 48 meses, y una duración aproximada del proyecto de cinco meses y medio. El coste imputado al proyecto se calcula de la siguiente forma:

$$\frac{1,200 \text{ €}}{48 \text{ meses}} \times 5,5 \text{ meses} = 137,50 \text{ €} \quad (\text{A.2})$$

Por tanto, el coste de hardware imputado al proyecto es de 137,50 €.

Costes de software y servicios

Las herramientas utilizadas durante el desarrollo no supusieron un coste económico directo. El proyecto se desarrolló con herramientas gratuitas o disponibles sin coste para el uso realizado, como GitHub, Vercel, Visual Studio Code, Node.js, npm, navegadores web y las librerías principales empleadas por la aplicación.

También se utilizaron herramientas de apoyo para pruebas, formateo, control de versiones y redacción de la memoria, sin necesidad de contratar planes de pago. Por tanto, el coste directo asociado al software y a los servicios empleados se considera nulo.

Coste total estimado

La Tabla A.2 resume el coste económico estimado del proyecto.

Concepto	Coste estimado
Coste de personal	10.695,00 €
Coste imputado de hardware	137,50 €
Costes de software y servicios	0,00 €
Costes adicionales	0,00 €
Total estimado	10.832,50 €

Tabla A.2: Coste total estimado del proyecto.

En conclusión, el proyecto fue viable económicamente dentro del contexto académico, ya que no requirió inversión directa en infraestructura ni contratación de servicios externos. La mayor parte del coste estimado corresponde al tiempo de desarrollo, análisis y documentación necesario para evolucionar una aplicación heredada de forma controlada.

Viabilidad legal

Desde el punto de vista legal, el proyecto se apoya en tecnologías y librerías de uso habitual en el desarrollo web. Al tratarse de la evolución de una aplicación existente, se revisó la licencia del proyecto original y las licencias de las principales dependencias utilizadas, con el objetivo de comprobar que la versión final pudiera distribuirse de forma coherente.

Licencias del software

La versión actual del proyecto se distribuye bajo licencia MIT. Esta licencia es adecuada para el contexto del TFG porque permite el uso, copia, modificación y distribución del código, manteniendo la obligación de conservar el aviso de copyright y el texto de la licencia.

Las dependencias principales utilizadas por la aplicación mantienen sus propias licencias. La Tabla A.3 resume las licencias más relevantes consideradas durante el desarrollo. En general, se trata de licencias permisivas, principalmente MIT [2] y Apache License 2.0 [3], compatibles con el uso académico y con la distribución del proyecto bajo licencia MIT.

Elemento	Licencia	Observación
Código desarrollado en el proyecto	MIT	La versión final incorpora un fichero <code>LICENSE</code> y la información correspondiente en <code>package.json</code> .
React	MIT	Librería principal empleada para construir la interfaz de usuario.
Material UI	MIT	Biblioteca de componentes utilizada para diferentes elementos de la interfaz.
mxGraph	Apache License 2.0	Librería utilizada para la creación y manipulación del lienzo de diagramas.
Vitest	MIT	Herramienta utilizada para la ejecución de pruebas unitarias.
Playwright	Apache License 2.0	Herramienta utilizada para la ejecución de pruebas end-to-end.
Biome	MIT	Herramienta empleada como apoyo para el análisis estático y el formateo del código.
Lefthook	MIT	Herramienta utilizada como apoyo en la automatización de comprobaciones durante el desarrollo.

Tabla A.3: Licencias principales consideradas en el proyecto.

Documentación, capturas y recursos gráficos

La memoria y los anexos se elaboran como documentación académica del Trabajo Fin de Grado. Las figuras utilizadas en la documentación corresponden principalmente a capturas propias de la aplicación, diagramas elaborados para explicar el proyecto o capturas de herramientas empleadas durante el desarrollo, como GitHub o Vercel.

En caso de incorporar recursos externos, estos deben citarse correctamente y respetar las condiciones de uso correspondientes. No obstante, el proyecto no depende de bancos de imágenes, vídeos de terceros ni datasets externos, por lo que los riesgos legales asociados a este tipo de recursos son reducidos.

Protección de datos

La aplicación funciona como herramienta cliente y no incorpora registro de usuarios, autenticación, backend propio ni envío de diagramas a un servidor. El almacenamiento del diagrama se realiza en el navegador del usuario o mediante archivos exportados cuando el propio usuario decide guardarlos.

Por este motivo, el proyecto no plantea un tratamiento centralizado de datos personales. Aun así, los diagramas creados o importados deben considerarse información bajo control del usuario, ya que pueden contener nombres o datos introducidos libremente durante el uso de la herramienta.

En conclusión, el proyecto resulta viable desde el punto de vista legal. Las licencias principales son compatibles con el uso académico y con la distribución del código bajo licencia MIT, no se utilizan recursos externos críticos y la aplicación no incorpora tratamiento remoto de datos personales.

Apéndice B

Especificación de Requisitos

B.1. Introducción

En este apartado se enumeran y desarrollan los objetivos y requisitos que debe tener la aplicación según el alcance final del proyecto. La herramienta parte de un editor de diagramas Entidad-Relación ya existente, por lo que los requisitos no se plantean como si el sistema se hubiera desarrollado desde cero, sino como una evolución de una aplicación heredada.

Durante el trabajo se han incorporado nuevas funcionalidades del modelo Entidad-Relación, se ha revisado la validación y la generación de SQL, y se han añadido mejoras de usabilidad para facilitar la edición de diagramas. Por ello, este anexo recoge tanto las funcionalidades principales del editor como las ampliaciones realizadas durante el desarrollo del TFG.

B.2. Objetivos generales

Desde el punto de vista de la especificación de requisitos, la aplicación debe conservar las funcionalidades principales del editor heredado y añadir las ampliaciones desarrolladas en este TFG de forma coherente con el modelo interno, la validación, la transformación relacional y la generación SQL. Los objetivos generales del sistema se resumen en los siguientes puntos:

- Modelar diagramas E-R mediante una aplicación web.
- Almacenar los diagramas en una estructura interna coherente y extensible.

- Representar elementos básicos y avanzados del modelo E-R dentro del alcance implementado.
- Validar la estructura interna del diagrama para detectar errores antes de generar resultados.
- Transformar el diagrama E-R validado a una representación relacional.
- Generar scripts SQL a partir de la representación relacional obtenida.
- Exportar e importar diagramas en formato JSON.
- Permitir la combinación controlada de diagramas importados o estructuras generadas.
- Exportar la representación visual del diagrama en formatos de imagen.
- Mejorar la usabilidad del editor sin alterar la notación académica del modelo E-R, incorporando acciones contextuales, selección múltiple visible y secciones de ayuda e información del proyecto.

B.3. Catálogo de requisitos

En este apartado se enumeran los requisitos funcionales y no funcionales que describen el comportamiento esperado de UBU E-R App tras la evolución realizada en este TFG. Los requisitos combinan funcionalidades heredadas que se mantienen como parte del editor, ampliaciones incorporadas durante el desarrollo y restricciones de alcance necesarias para no atribuir al sistema capacidades que todavía no implementa.

Requisitos funcionales

- **RF1 - Modelar elementos básicos E-R:** La aplicación debe permitir crear y manipular entidades, atributos y relaciones dentro del lienzo del editor.
- **RF2 - Configurar atributos:** La aplicación debe permitir trabajar con atributos simples, clave, compuestos, multivaluados y discriminantes de entidades débiles en los casos soportados.
- **RF3 - Configurar relaciones:** La aplicación debe permitir configurar relaciones binarias, cardinalidades y atributos propios de relación cuando proceda.
- **RF4 - Modelar relaciones ternarias y roles:** La aplicación debe permitir representar relaciones de grado tres y distinguir roles de participación cuando sean necesarios.
- **RF5 - Modelar entidades débiles:** La aplicación debe permitir representar entidades débiles, relaciones identificadoras y discriminantes.

- **RF6 - Modelar jerarquías ISA sencillas:** La aplicación debe permitir representar una generalización con una o varias especializaciones.
- **RF7 - Acotar el soporte de jerarquías ISA:** La aplicación debe limitar el tratamiento de ISA al caso implementado. En particular, no debe tratar especializaciones con clave propia, herencia múltiple real, discriminantes ISA ni restricciones de totalidad, parcialidad, solapamiento o exclusividad.
- **RF8 - Mantener el modelo interno:** La aplicación debe mantener una estructura interna coherente con el contenido representado en el lienzo.
- **RF9 - Persistir diagramas localmente:** La aplicación debe conservar el diagrama en el almacenamiento local del navegador.
- **RF10 - Exportar e importar JSON:** La aplicación debe permitir exportar diagramas a JSON e importar diagramas desde archivos JSON válidos.
- **RF11 - Reemplazar o combinar contenido:** La aplicación debe permitir importar diagramas o generar estructuras reemplazando el diagrama actual o combinándolas con el contenido existente.
- **RF12 - Validar diagramas:** La aplicación debe validar el modelo interno y mostrar los errores detectados de forma comprensible.
- **RF13 - Generar SQL:** La aplicación debe transformar el diagrama validado al modelo relacional y generar un script SQL con un formato claro y legible.
- **RF14 - Generar estructuras básicas:** La aplicación debe permitir crear estructuras de ejemplo para facilitar la prueba y exploración del editor.
- **RF15 - Exportar imagen del diagrama:** La aplicación debe permitir exportar la representación visual del diagrama en formato PNG y SVG.
- **RF16 - Ofrecer acciones de apoyo:** La aplicación debe incluir acciones como comprobar el diagrama, ajustar la vista, cambiar el idioma de la interfaz, consultar ayuda, visualizar información del proyecto y reiniciar el lienzo previa confirmación.
- **RF17 - Gestionar selección múltiple y acciones contextuales:** La aplicación debe permitir seleccionar varios elementos de forma visible, mover o eliminar una selección múltiple y evitar mostrar acciones individuales cuando hay varios elementos seleccionados.

El soporte de ISA se considera inicial y controlado: se contempla una única generalización con una o varias especializaciones, con clave heredada desde la generalización, pero quedan fuera del alcance la totalidad o parcialidad,

el solapamiento o exclusividad, los discriminantes ISA y la herencia múltiple real. Las agregaciones no forman parte de los requisitos implementados en esta versión.

Requisitos no funcionales

- **RNF1 - Usabilidad:** La aplicación debe ofrecer una interfaz clara y cómoda para la creación y revisión de diagramas E-R, incluyendo guías, descripciones breves de acciones, acciones contextuales y retroalimentación visual durante la selección de elementos.
- **RNF2 - Compatibilidad:** La aplicación debe poder ejecutarse en navegadores web modernos.
- **RNF3 - Mantenibilidad:** El código debe organizarse de forma que permita seguir ampliando el editor con nuevas funcionalidades.
- **RNF4 - Coherencia visual:** La representación gráfica debe respetar la notación académica utilizada para entidades, relaciones, atributos, entidades débiles, relaciones identificadoras e ISA.
- **RNF5 - Separación de responsabilidades:** La aplicación debe mantener una separación razonable entre interfaz, modelo interno, validación, transformación relacional y generación SQL.
- **RNF6 - Funcionamiento sin backend:** La aplicación debe funcionar sin requerir autenticación de usuarios, servidor propio ni almacenamiento remoto.
- **RNF7 - Contexto académico:** La herramienta debe servir como apoyo práctico para el aprendizaje y uso del modelo Entidad-Relación y su transformación relacional.

B.4. Especificación de requisitos

Los requisitos funcionales se agrupan en varios bloques. El primero corresponde a la edición visual del diagrama, que incluye la creación y modificación de entidades, atributos y relaciones. El segundo bloque recoge los elementos avanzados del modelo E-R soportados por la aplicación, como entidades débiles, atributos compuestos y multivaluados, relaciones ternarias, roles y jerarquías ISA sencillas.

Otro bloque importante es el mantenimiento del modelo interno. Cada elemento representado en el lienzo debe conservarse en una estructura de datos que permita reconstruir el diagrama, validarlo, persistirlo y transformarlo

posteriormente. Esta estructura es también la base para la exportación e importación en formato JSON.

La validación del diagrama permite comprobar si el modelo contiene errores antes de generar una salida relacional o SQL. Además, el usuario puede ejecutar una comprobación explícita del diagrama sin necesidad de generar SQL. Si se detectan errores, la aplicación debe mostrarlos de forma agrupada y comprensible.

La transformación relacional y la generación de SQL constituyen la salida principal del sistema. A partir del diagrama validado, la aplicación obtiene una representación basada en tablas, columnas, claves primarias y claves foráneas, y posteriormente genera el script SQL correspondiente.

Finalmente, la aplicación incorpora varias funcionalidades de apoyo a la edición: generación de estructuras básicas, importación y generación combinada de contenido, exportación visual en PNG y SVG, ajuste de la vista, soporte básico de idioma, selección múltiple visible, acciones contextuales, secciones de ayuda e información del proyecto y reinicio controlado del lienzo.

Actores

El sistema contempla un actor principal, denominado usuario del editor. Este actor representa a la persona que utiliza UBU E-R App para crear, revisar, validar, importar, exportar o transformar diagramas Entidad-Relación. En el contexto académico del proyecto, este usuario puede corresponderse con un estudiante, un docente o cualquier persona que necesite modelar un esquema E-R y obtener una representación relacional o SQL a partir de él.

Diagrama de casos de uso

La figura [B.1](#) resume los principales casos de uso identificados para UBU E-R App. Se ha considerado un único actor principal, el usuario del editor, ya que la aplicación no incorpora autenticación, perfiles diferenciados ni backend propio. Los casos de uso se agrupan en edición del modelo E-R, validación y generación, persistencia e intercambio, y acciones de apoyo a la edición.

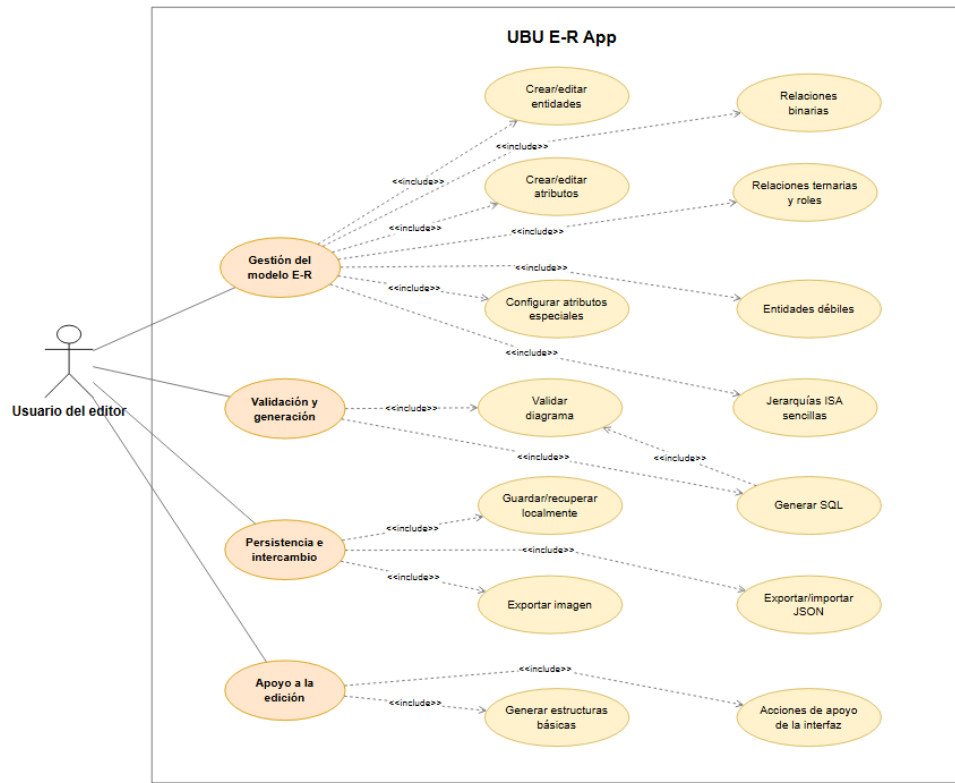


Figura B.1: Diagrama de casos de uso de UBU E-R App.

Casos de uso

Las tablas siguientes describen los principales casos de uso del sistema a nivel funcional, sin entrar en detalles internos de implementación ni en pasos concretos dependientes de la interfaz.

CU-1	Crear y editar entidades
Versión	1.0
Autor	Steven Paul Alba Alba
Requisitos asociados	RF1, RF8
Descripción	Permite al usuario crear entidades en el lienzo y modificar sus propiedades básicas.
Precondición	La aplicación debe estar abierta y funcionando.
Acciones	1. El usuario crea una entidad en el lienzo. 2. El usuario puede mover, seleccionar, renombrar o eliminar la entidad. 3. Si se trata de una entidad fuerte normal, la aplicación puede crear una clave <code>id</code> por defecto.
Postcondición	La entidad queda representada visualmente y almacenada en el modelo interno.
Excepciones	-
Importancia	Alta

Tabla B.1: CU-1 Crear y editar entidades.

CU-2	Crear y editar atributos
Versión	1.0
Autor	Steven Paul Alba Alba
Requisitos asociados	RF1, RF2, RF8
Descripción	Permite añadir atributos a entidades o relaciones y modificar su información básica.
Precondición	Debe existir una entidad o relación seleccionable en el diagrama.
Acciones	1. El usuario selecciona el elemento al que desea añadir un atributo. 2. El usuario crea el atributo. 3. El usuario puede renombrar, mover o eliminar el atributo.
Postcondición	El atributo queda asociado al elemento correspondiente en el lienzo y en el modelo interno.
Excepciones	El atributo no se crea si no existe un elemento válido al que asociarlo.
Importancia	Alta

Tabla B.2: CU-2 Crear y editar atributos.

CU-3	Configurar atributos especiales
Versión	1.0
Autor	Steven Paul Alba Alba
Requisitos asociados	RF2
Descripción	Permite configurar atributos con significado especial dentro del modelo E-R.
Precondición	Debe existir al menos un atributo asociado a una entidad o relación.
Acciones	1. El usuario selecciona un atributo. 2. El usuario configura el atributo como clave, discriminante de entidad débil, compuesto o multi-valuado cuando la acción está disponible. 3. En atributos compuestos, el usuario puede añadir subatributos.
Postcondición	El atributo queda representado con la notación correspondiente y su tipo se conserva en el modelo interno.
Excepciones	Algunas acciones no están disponibles si el atributo pertenece a un elemento donde esa configuración no es válida.
Importancia	Alta

Tabla B.3: CU-3 Configurar atributos especiales.

CU-4	Crear y configurar relaciones binarias
Versión	1.0
Autor	Steven Paul Alba Alba
Requisitos asociados	RF3, RF8
Descripción	Permite crear relaciones binarias entre entidades y configurar sus participantes y cardinalidades.
Precondición	Deben existir entidades en el diagrama.
Acciones	1. El usuario crea una relación. 2. El usuario selecciona las entidades participantes. 3. El usuario configura las cardinalidades de la relación. 4. Si procede, el usuario añade atributos propios a la relación.
Postcondición	La relación queda conectada a sus entidades participantes y almacenada en el modelo interno.
Excepciones	La relación puede quedar pendiente de validación si faltan participantes o cardinalidades.
Importancia	Alta

Tabla B.4: CU-4 Crear y configurar relaciones binarias.

CU-5	Crear y configurar relaciones ternarias y roles
Versión	1.0
Autor	Steven Paul Alba Alba
Requisitos asociados	RF4, RF8
Descripción	Permite representar relaciones de grado tres y distinguir roles de participación cuando sea necesario.
Precondición	Deben existir entidades suficientes para configurar una relación ternaria.
Acciones	1. El usuario crea una relación ternaria. 2. El usuario selecciona sus tres participantes. 3. El usuario configura cardinalidades. 4. El usuario define roles cuando una entidad participa más de una vez o cuando se requiere mayor claridad.
Postcondición	La relación ternaria queda almacenada con sus participantes, cardinalidades y roles.
Excepciones	La validación informa de participantes, cardinalidades o roles incorrectos cuando proceda.
Importancia	Alta

Tabla B.5: CU-5 Crear y configurar relaciones ternarias y roles.

CU-6	Modelar entidades débiles y relaciones identificadoras
Versión	1.0
Autor	Steven Paul Alba Alba
Requisitos asociados	RF5, RF8, RF13
Descripción	Permite representar una entidad débil y su dependencia por identificación respecto a una entidad propietaria.
Precondición	Deben existir las entidades necesarias para configurar la dependencia.
Acciones	1. El usuario marca una entidad como débil. 2. El usuario configura la relación identificadora. 3. El usuario establece la entidad propietaria. 4. El usuario define el discriminante correspondiente.
Postcondición	La entidad débil queda representada con la notación adecuada y se tiene en cuenta en la transformación relacional.
Excepciones	La validación informa si la entidad débil no tiene propietario, relación identificadora o discriminante válido.
Importancia	Alta

Tabla B.6: CU-6 Modelar entidades débiles y relaciones identificadoras.

CU-7	Modelar jerarquías ISA sencillas
Versión	1.0
Autor	Steven Paul Alba Alba
Requisitos asociados	RF6, RF7, RF8, RF13
Descripción	Permite representar una jerarquía ISA con una entidad general y una o varias especializaciones.
Precondición	Deben existir las entidades que participarán en la jerarquía.
Acciones	1. El usuario crea un elemento ISA. 2. El usuario selecciona la entidad general. 3. El usuario añade una o varias entidades especializadas. 4. La aplicación impide configuraciones fuera del alcance implementado.
Postcondición	La jerarquía ISA queda representada en el lienzo y almacenada en el modelo interno.
Excepciones	La validación informa de ISA sin generalización, sin especializaciones, con claves propias en especializaciones o con configuraciones de herencia no soportadas. Las restricciones de totalidad o parcialidad, solapamiento o exclusividad y discriminante ISA quedan fuera del alcance.
Importancia	Alta

Tabla B.7: CU-7 Modelar jerarquías ISA sencillas.

CU-8	Validar explícitamente el diagrama
Versión	1.0
Autor	Steven Paul Alba Alba
Requisitos asociados	RF12
Descripción	Permite comprobar si el diagrama actual es válido sin generar SQL ni exportar información.
Precondición	La aplicación debe estar abierta y puede existir un diagrama completo o en construcción.
Acciones	1. El usuario selecciona la acción de comprobar el diagrama. 2. La aplicación ejecuta las reglas de validación sobre el modelo interno. 3. La aplicación muestra el resultado de la validación.
Postcondición	Si el diagrama es válido, se muestra una notificación de éxito. Si contiene errores, se abre el diálogo de validación.
Excepciones	-
Importancia	Alta

Tabla B.8: CU-8 Validar explícitamente el diagrama.

CU-9	Generar SQL a partir del diagrama
Versión	1.0
Autor	Steven Paul Alba Alba
Requisitos asociados	RF12, RF13
Descripción	Permite generar un script SQL a partir del diagrama E-R validado.
Precondición	El diagrama debe contener una estructura suficiente para poder validarse.
Acciones	1. El usuario selecciona la opción de generar SQL. 2. La aplicación valida el diagrama. 3. La aplicación transforma el modelo E-R a una representación relacional. 4. La aplicación genera el script SQL correspondiente.
Postcondición	Si el diagrama es válido, se obtiene el SQL generado. Si no lo es, se muestran los errores detectados.
Excepciones	Errores de validación del diagrama.
Importancia	Alta

Tabla B.9: CU-9 Generar SQL a partir del diagrama.

CU-10	Guardar y recuperar el diagrama localmente
Versión	1.0
Autor	Steven Paul Alba Alba
Requisitos asociados	RF8, RF9
Descripción	Permite conservar el estado del diagrama entre recargas o sesiones del navegador.
Precondición	El usuario debe haber creado o modificado un diagrama.
Acciones	1. La aplicación mantiene actualizado el modelo interno. 2. La aplicación guarda el estado en el almacenamiento local del navegador. 3. Al recargar, la aplicación reconstruye el diagrama guardado.
Postcondición	El usuario puede continuar trabajando sobre el diagrama sin perder el contenido por una recarga.
Excepciones	El contenido puede no recuperarse si se borra el almacenamiento local del navegador.
Importancia	Alta

Tabla B.10: CU-10 Guardar y recuperar el diagrama localmente.

CU-11	Exportar e importar diagramas en JSON
Versión	1.0
Autor	Steven Paul Alba Alba
Requisitos asociados	RF10, RF11
Descripción	Permite guardar el diagrama en un archivo JSON o cargar un diagrama previamente exportado.
Precondición	Para exportar, debe existir un diagrama. Para importar, el usuario debe disponer de un archivo JSON compatible.
Acciones	1. El usuario selecciona la opción de exportar o importar JSON. 2. En la importación, el usuario elige entre reemplazar el diagrama actual o combinar el contenido importado. 3. La aplicación valida y reconstruye el resultado cuando procede.
Postcondición	El diagrama se guarda como JSON o se carga en el editor manteniendo una estructura interna coherente.
Excepciones	Archivo incompatible, JSON inválido o diagrama combinado no válido.
Importancia	Media

Tabla B.11: CU-11 Exportar e importar diagramas en JSON.

CU-12	Generar estructuras básicas de ejemplo
Versión	1.0
Autor	Steven Paul Alba Alba
Requisitos asociados	RF11, RF14
Descripción	Permite crear de forma rápida estructuras habituales del modelo E-R.
Precondición	La aplicación debe estar abierta y funcionando.
Acciones	1. El usuario abre la acción de generar estructura. 2. El usuario selecciona una estructura básica. 3. El usuario decide si desea reemplazar el diagrama actual o combinar la estructura con el contenido existente.
Postcondición	La estructura seleccionada aparece en el lienzo y queda integrada en el modelo interno.
Excepciones	La operación no se realiza si el usuario cancela la confirmación. Si el diagrama resultante queda incompleto o incoherente, podrá detectarse posteriormente mediante la validación del diagrama.
Importancia	Media

Tabla B.12: CU-12 Generar estructuras básicas de ejemplo.

CU-13	Exportar la representación visual del diagrama
Versión	1.0
Autor	Steven Paul Alba Alba
Requisitos asociados	RF15
Descripción	Permite obtener una imagen del diagrama para documentación, revisión o uso externo.
Precondición	Debe existir contenido visual en el lienzo del editor.
Acciones	1. El usuario selecciona la opción de exportar imagen. 2. El usuario elige el formato disponible, PNG o SVG. 3. La aplicación genera el archivo visual correspondiente.
Postcondición	El usuario obtiene una imagen del diagrama. Esta operación no modifica el modelo interno ni el SQL generado.
Excepciones	La aplicación informa si no existe contenido suficiente para exportar.
Importancia	Media

Tabla B.13: CU-13 Exportar la representación visual del diagrama.

CU-14	Utilizar acciones de apoyo de la interfaz
Versión	1.0
Autor	Steven Paul Alba Alba
Requisitos asociados	RF16, RF17
Descripción	Permite utilizar acciones auxiliares que facilitan la interacción con el editor.
Precondición	La aplicación debe estar abierta.
Acciones	1. El usuario puede ajustar la vista al contenido del diagrama. 2. El usuario puede cambiar el idioma de la interfaz entre español e inglés. 3. El usuario puede consultar la guía mostrada cuando no hay selección. 4. El usuario puede reiniciar el lienzo previa confirmación. 5. El usuario puede consultar la ayuda de uso y la información del proyecto. 6. El usuario puede seleccionar varios elementos de forma visible. 7. El usuario puede mover o eliminar una selección múltiple cuando proceda.
Postcondición	La edición del diagrama resulta más cómoda sin modificar la semántica del modelo E-R.
Excepciones	El reinicio del lienzo no se realiza si el usuario cancela la confirmación.
Importancia	Media

Tabla B.14: CU-14 Utilizar acciones de apoyo de la interfaz.

Apéndice C

Especificación de diseño

C.1. Introducción

En este anexo se describe el diseño general de la aplicación resultante de la evolución realizada durante el proyecto. El objetivo no es recoger todos los detalles internos del código, sino explicar cómo se organiza el editor y cómo se relacionan sus partes principales.

La aplicación parte de un proyecto heredado, por lo que el diseño final es el resultado de comprender una base existente y adaptarla progresivamente. El trabajo no consistió únicamente en añadir elementos visuales al lienzo, sino en mantener la coherencia entre la representación gráfica, la representación interna del diagrama, la validación, la transformación relacional y la generación de SQL.

De forma general, el sistema puede entenderse como una aplicación web compuesta por tres bloques principales. El primero es la interfaz visual, que permite al usuario crear y modificar el diagrama. El segundo es la representación interna o metamodelo, que almacena la información estructurada del diagrama. El tercero está formado por los procesos que trabajan sobre esa representación: validación, transformación relacional, generación de SQL, persistencia, importación y exportación.

C.2. Diseño de datos

El diseño de datos de la aplicación se centra en el metamodelo utilizado para representar los diagramas Entidad-Relación creados por el usuario. La herramienta no utiliza una base de datos propia ni un servidor que

almacene los diagramas, pero sí mantiene una representación estructurada del contenido del lienzo. Esta representación es la base sobre la que se apoyan la edición, la validación, la persistencia local, la importación y exportación en JSON, la transformación al modelo relacional y la generación de SQL.

En este contexto, se utiliza el término metamodelo porque la aplicación no modela directamente un dominio concreto, como una empresa, una biblioteca o un sistema de ventas, sino los elementos que pueden aparecer dentro de un diagrama E/R. Es decir, el sistema almacena información sobre entidades, atributos, relaciones, participantes, cardinalidades y jerarquías ISA. A partir de esa información puede reconstruir el diagrama visual y aplicar las operaciones propias del editor.

Metamodelo del diagrama

La Figura C.1 muestra una representación conceptual del metamodelo utilizado por la aplicación. Este diagrama no representa una base de datos física, sino la estructura lógica que permite describir los diagramas creados en el editor y conservar la información necesaria para procesarlos posteriormente.

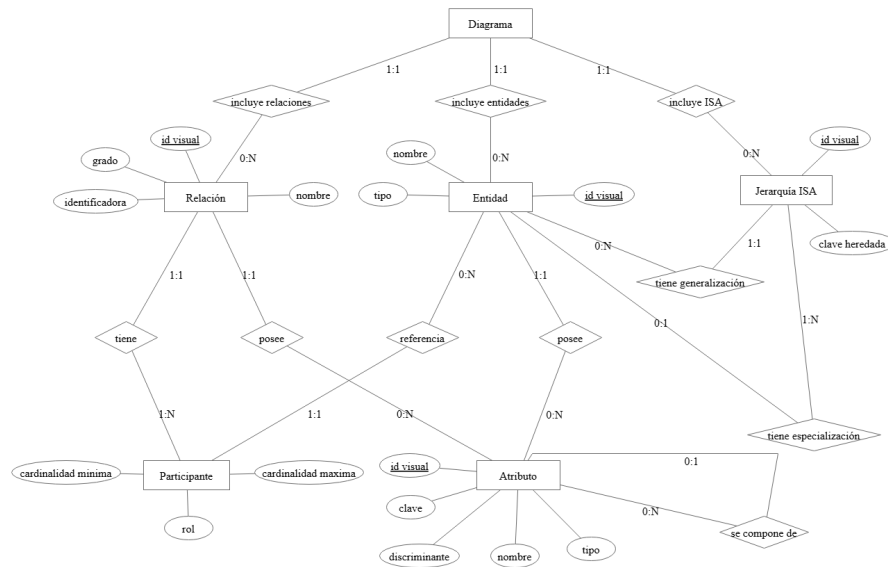


Figura C.1: Metamodelo E/R de la representación interna de un diagrama.

El elemento principal del metamodelo es el diagrama, que actúa como contenedor de los elementos creados por el usuario. Dentro de un diagrama

pueden existir entidades, relaciones y jerarquías ISA. Las entidades representan los conjuntos de entidades del modelo E/R creado por el usuario, mientras que las relaciones representan las interrelaciones existentes entre ellas.

Las relaciones no enlazan directamente todos sus datos con las entidades, sino que incorporan participantes. Cada participante vincula una relación con una entidad concreta y conserva información propia de esa participación, como la cardinalidad mínima, la cardinalidad máxima y, cuando procede, el rol. Esta separación resulta especialmente útil para representar relaciones ternarias y casos en los que una misma entidad participa más de una vez en una relación con distintos roles. La comprobación de que una relación dispone de los participantes necesarios no se expresa únicamente mediante la cardinalidad del metamodelo, sino también mediante las reglas de validación del editor.

Los atributos se tratan como elementos propios del metamodelo porque pueden presentar distintas propiedades relevantes para la validación y la transformación posterior. Entre ellas se encuentran su condición de clave, atributo compuesto, atributo multivaluado o discriminante, según el tipo de elemento y el alcance soportado por la aplicación.

En el metamodelo, un atributo puede estar asociado a una entidad, a una relación o a otro atributo cuando forma parte de una estructura compuesta. Estas asociaciones representan alternativas de pertenencia dentro del editor, no la obligación de que un mismo atributo aparezca simultáneamente en todos esos contextos.

Las jerarquías ISA se representan mediante una entidad general y un conjunto de entidades especializadas. El alcance implementado contempla una generalización con una o varias especializaciones y clave heredada desde la entidad general. No se incluyen en el metamodelo implementado características como totalidad o parcialidad, solapamiento o exclusividad, discriminantes ISA ni herencia múltiple real, que quedan fuera del alcance desarrollado.

Correspondencia con la representación JSON

La representación JSON exportada por la aplicación sigue la organización definida por el metamodelo. El archivo generado almacena las colecciones de entidades, relaciones, atributos, jerarquías ISA y referencias necesarias para reconstruir el diagrama. Por tanto, el JSON no debe entenderse como

una base de datos independiente, sino como una serialización del estado del editor.

Cada elemento conserva identificadores visuales que permiten relacionar la información lógica con su representación en el lienzo gestionado mediante mxGraph. Esta correspondencia es necesaria para reconstruir el diagrama tras una importación o después de recuperar el estado guardado en el almacenamiento local del navegador. De esta forma, la aplicación puede volver a crear las celdas visuales, las conexiones y los elementos auxiliares manteniendo la coherencia entre el metamodelo del diagrama y su representación gráfica.

La Figura C.2 resume esta correspondencia entre el metamodelo, la representación JSON y los procesos que trabajan sobre ella.

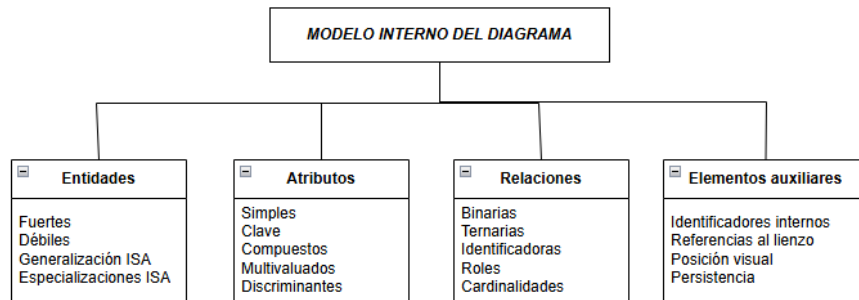


Figura C.2: Correspondencia simplificada entre el metamodelo, la representación JSON y los procesos de la aplicación.

La persistencia local y la exportación a JSON reutilizan esta misma estructura. Cuando el usuario exporta un diagrama, la aplicación genera un archivo que contiene la información necesaria para volver a cargarlo. Cuando importa un archivo, el sistema interpreta esa estructura y reconstruye tanto el contenido lógico como los elementos visuales asociados. En las operaciones de combinación, el contenido importado se inserta de forma controlada para evitar conflictos directos de identificadores y facilitar la revisión posterior en el lienzo.

Diccionario de datos

A continuación se recoge un diccionario de datos simplificado del metamodelo. El objetivo no es enumerar todos los campos internos de implementación, sino describir el significado de los principales elementos almacenados por la aplicación y su papel dentro del editor.

Elemento	Descripción
Diagrama	Contenedor principal del estado del editor. Agrupa las entidades, relaciones, atributos, jerarquías ISA y referencias necesarias para reconstruir el contenido del lienzo.
Entidad	Representa un conjunto de entidades del modelo E/R creado por el usuario. Conserva su nombre, su identificador visual y propiedades relevantes para la edición, como su posible condición de entidad débil, generalización o especialización ISA.
Relación	Representa una interrelación entre entidades. Conserva su nombre, identificador visual, grado, participantes, cardinalidades y propiedades específicas, como su posible condición de relación identificadora.
Participante	Elemento que vincula una relación con una entidad concreta. Permite conservar la cardinalidad mínima, la cardinalidad máxima y, en relaciones con entidades repetidas, el rol que diferencia cada participación.
Atributo	Representa una propiedad asociada a una entidad, a una relación o a otro atributo en el caso de atributos compuestos. Puede contener información sobre clave, discriminante, atributos compuestos, atributos multivaluados y subatributos.
Jerarquía ISA	Representa una estructura de generalización/especialización. Conserva la entidad general y las entidades especializadas, manteniendo la restricción de clave heredada dentro del alcance implementado.
Identificador visual	Referencia que permite mantener la correspondencia entre los elementos del metamodelo y las celdas del lienzo gestionadas mediante mxGraph. Es necesaria para reconstruir el diagrama y sincronizar los cambios visuales con la representación interna.

Tabla C.1: Diccionario de datos simplificado del metamodelo del diagrama.

Este diseño permite separar, al menos conceptualmente, el contenido lógico del diagrama y su representación visual. No obstante, al tratarse

de una aplicación heredada basada en mxGraph, existe un acoplamiento práctico entre ambos niveles durante la sincronización de elementos. Por esta razón, parte del trabajo realizado se orientó a mantener la coherencia entre el lienzo, la representación interna, la validación, la transformación relacional y la generación de SQL sin introducir una reescritura completa del editor.

C.3. Diseño arquitectónico

El diseño arquitectónico describe la organización general de la aplicación y la separación entre los bloques que intervienen en la edición, validación, transformación y persistencia de los diagramas. La herramienta se desarrolla como una aplicación web de cliente, por lo que la mayor parte de la lógica se ejecuta en el navegador del usuario.

La aplicación no dispone de un servidor propio encargado de procesar los diagramas ni de una base de datos remota asociada al proyecto. El despliegue en Vercel se utiliza para servir la aplicación web generada durante el proceso de construcción. Una vez cargada en el navegador, la herramienta trabaja localmente con el lienzo de mxGraph, la representación interna del diagrama, el almacenamiento local del navegador y los archivos importados o exportados por el usuario.

Diagrama de despliegue

La Figura C.3 muestra el despliegue general de la aplicación. El usuario accede desde un navegador web a la versión publicada de UBU E-R App. Vercel sirve los recursos estáticos generados a partir de la aplicación React, pero no ejecuta la lógica de validación ni la transformación de los diagramas en un servidor propio. Estas operaciones se realizan en el cliente.

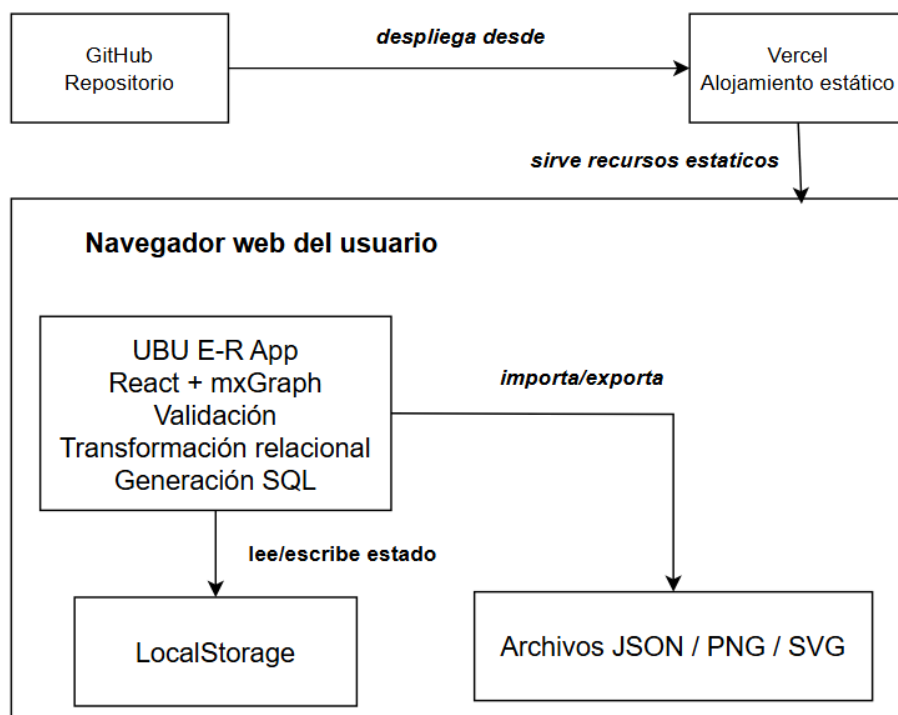


Figura C.3: Diagrama UML de despliegue de la aplicación.

En este despliegue pueden distinguirse tres elementos principales. En primer lugar, el repositorio GitHub, utilizado como sistema de control de versiones y como origen del código fuente. En segundo lugar, Vercel, que construye y publica la aplicación web a partir del repositorio. En tercer lugar, el navegador del usuario, donde se ejecuta la aplicación, se manipula el lienzo y se almacena temporalmente el estado del diagrama mediante *LocalStorage*.

La importación y exportación de diagramas se realiza mediante archivos JSON gestionados desde el propio navegador. Del mismo modo, la exportación visual genera archivos de imagen a partir del contenido del lienzo. Por tanto, estas operaciones no requieren una base de datos externa ni un servicio backend específico.

Diagrama de componentes

La Figura C.4 resume los principales componentes lógicos de la aplicación y sus dependencias. El componente central es el editor de diagramas, que

coordina la interfaz React, el lienzo gestionado mediante mxGraph, la representación interna del diagrama y los módulos de dominio.

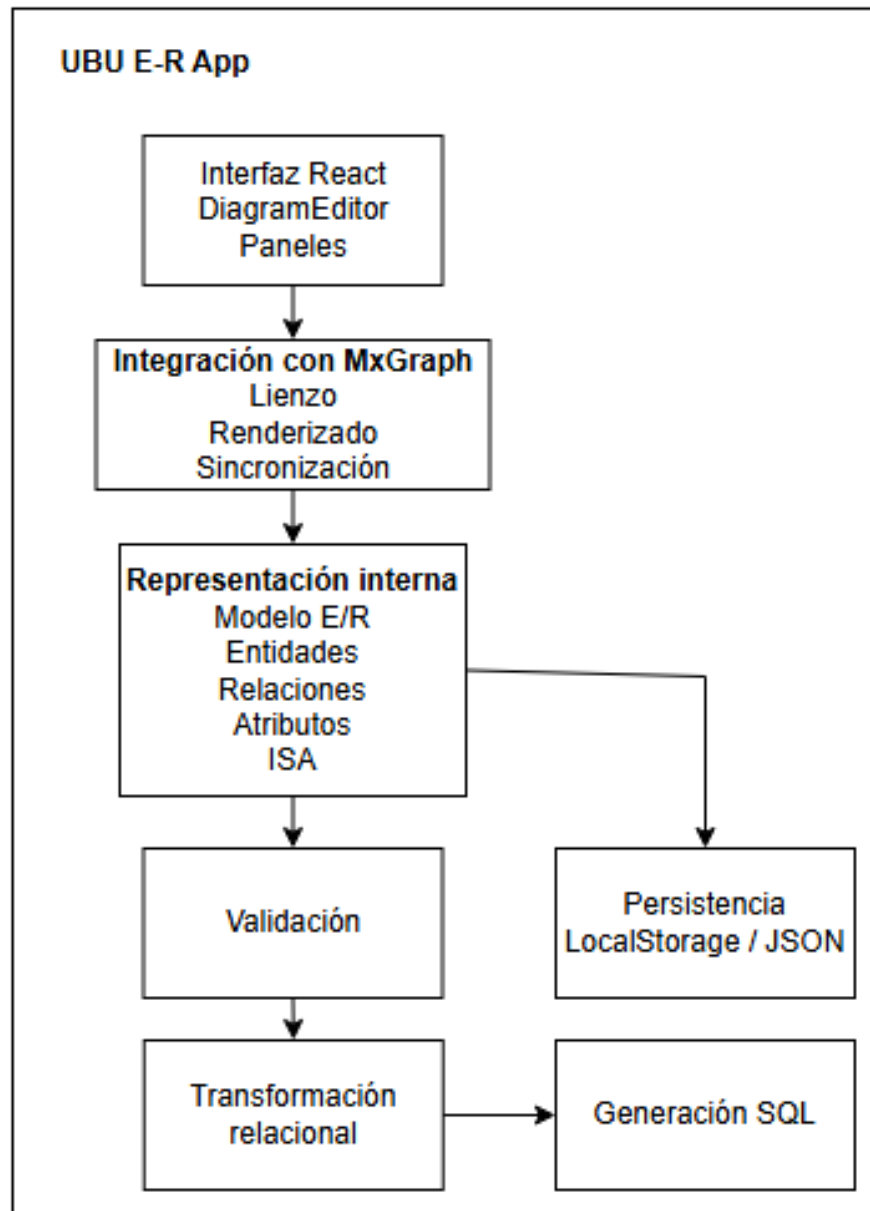


Figura C.4: Diagrama UML de componentes de la aplicación.

La interfaz de usuario se organiza alrededor del componente principal del editor y de los paneles de acciones. Estos paneles permiten crear elementos

E/R, ejecutar operaciones globales sobre el diagrama y mostrar acciones contextuales en función de la selección actual. La interacción visual con el lienzo se apoya en `mxGraph`, que proporciona la infraestructura necesaria para representar y manipular celdas, conexiones y elementos gráficos.

La lógica del editor se apoya en varios grupos de módulos. Los *hooks* del editor agrupan acciones relacionadas con historial, persistencia, atributos, relaciones, ISA y borrado. Las utilidades específicas del editor encapsulan operaciones de integración con `mxGraph`, renderizado, sincronización, selección, persistencia y reconstrucción visual. Por su parte, los módulos de dominio del modelo E/R concentran reglas y funciones independientes de la interfaz siempre que es posible.

Sobre la representación interna del diagrama trabajan los módulos de validación, transformación relacional y generación de SQL. La validación comprueba que el diagrama sea coherente dentro del alcance implementado. La transformación relacional construye una representación intermedia basada en tablas, columnas, claves primarias y claves foráneas. Finalmente, el servicio de generación de SQL convierte esa representación en un script compatible con el enfoque simplificado adoptado por la aplicación.

Organización interna del código

La estructura del código fuente refleja esta separación de responsabilidades. El componente `DiagramEditor.js` actúa como orquestador principal del editor. En él se conservan la inicialización de `mxGraph`, las referencias principales, los efectos de ciclo de vida y la composición de los módulos extraídos.

Las acciones específicas del editor se agrupan en *hooks*, como la gestión del historial, la persistencia, las acciones de atributos, relaciones, ISA y borrado. La interfaz contextual y las acciones globales se organizan en paneles diferenciados. Las utilidades del editor se dividen según su responsabilidad principal: integración con `mxGraph`, renderizado, sincronización, persistencia, selección y validación orientada a la interfaz.

Fuera del componente visual, los módulos de dominio se organizan en torno al modelo E/R y al modelo relacional. La carpeta de dominio E/R agrupa reglas y funciones relacionadas con entidades, relaciones, atributos, ISA, composición de diagramas y validación. La parte relacional contiene la lógica de transformación desde el diagrama E/R hacia una representación basada en tablas. La generación final de SQL se mantiene en servicios

específicos, separando la transformación conceptual del renderizado textual del script.

Mantenibilidad y extensibilidad

La arquitectura final no plantea una reescritura completa de la aplicación heredada. La evolución se realizó de forma incremental, separando responsabilidades allí donde aportaba claridad sin introducir riesgos innecesarios. Por este motivo, algunas partes especialmente acopladas a mxGraph, como la inicialización del lienzo y determinados efectos de sincronización, permanecen en el componente principal del editor.

Esta organización facilita que futuras ampliaciones puedan localizar mejor el punto de cambio. Las reglas del dominio E/R deberían incorporarse en los módulos de dominio, las transformaciones hacia el modelo relacional en la parte relacional, la generación textual en los servicios SQL, las acciones de edición en los *hooks* del editor y los cambios de interfaz en los paneles correspondientes.

Aun así, al tratarse de una aplicación heredada basada en mxGraph, sigue existiendo una dependencia práctica entre la representación visual y la representación interna durante la sincronización de elementos. Por ello, cualquier ampliación futura debería mantener la coherencia entre el lienzo, el metamodelo, la validación, la transformación relacional, la persistencia y la generación de SQL.

C.4. Diseño procedimental

El diseño procedimental recoge los flujos de funcionamiento más relevantes de la aplicación. No se pretende describir cada interacción posible de la interfaz, sino mostrar cómo las acciones principales del usuario se traducen en cambios sobre la representación interna del diagrama y cómo dicha representación se valida y transforma para generar resultados.

Flujo general de edición

El flujo principal comienza cuando el usuario crea o modifica elementos en el lienzo del editor. Cada acción visual debe reflejarse en la representación interna, de forma que lo que aparece en pantalla coincida con la información que la aplicación conserva y procesa.

Este flujo incluye operaciones como crear entidades, añadir atributos, configurar relaciones, establecer cardinalidades, mover elementos, renombrarlos o eliminarlos. En todos los casos, la aplicación debe mantener la sincronización entre la vista y la representación interna del diagrama.

Además, la interfaz contempla la selección múltiple de elementos. Este comportamiento permite mover o eliminar varios elementos de forma conjunta, pero evita mostrar acciones individuales cuando la selección contiene más de un elemento. De esta forma, se mantiene la coherencia entre las acciones disponibles en la interfaz y las operaciones que realmente pueden aplicarse sobre el modelo.

Validación del modelo

Antes de generar una salida relacional o SQL, la aplicación valida la representación interna del diagrama. Esta validación comprueba que el diagrama contiene la información necesaria y que no existen configuraciones incompletas o incompatibles con el alcance implementado.

La validación se ejecuta en operaciones como la generación de SQL y también puede lanzarse de forma explícita mediante la acción de comprobar el diagrama. Cuando se detectan errores, la aplicación debe mostrarlos de forma comprensible para que el usuario pueda corregir el modelo.

En el caso de ISA, la validación impide configuraciones fuera del alcance implementado, como una ISA sin generalización, una ISA sin especializaciones, especializaciones con clave propia o entidades especializadas en más de una jerarquía.

Transformación al modelo relacional

Si el diagrama supera la validación, la aplicación transforma el modelo Entidad-Relación a una representación relacional. Esta representación se basa en tablas, columnas, claves primarias y claves foráneas.

La existencia de una representación intermedia permite separar la lógica de transformación de la generación concreta del script SQL. De esta forma, la aplicación primero decide qué tablas y restricciones deben existir, y después construye el texto SQL correspondiente.

Generación de SQL

La generación de SQL toma como entrada la representación relacional obtenida en la fase anterior. A partir de ella, se generan las sentencias necesarias para crear las tablas y definir sus claves principales y foráneas.

En la versión final se opta por una salida SQL más legible, evitando nombres explícitos de restricciones de clave foránea cuando no resultan necesarios y simplificando las referencias cuando una clave foránea referencia la clave primaria completa de la tabla destino. Esta decisión no modifica la transformación relacional previa, sino únicamente la forma en la que se renderiza el script SQL generado.

La salida SQL debe entenderse como una traducción razonable del modelo relacional generado por la aplicación, no como un generador industrial completo para cualquier sistema gestor de bases de datos. Su objetivo principal dentro del proyecto es servir como apoyo académico y como punto de partida para comprender la transformación desde el modelo Entidad-Relación.

La Figura C.5 resume este proceso: la representación interna se valida antes de construir la representación relacional y solo después se genera el script SQL correspondiente.

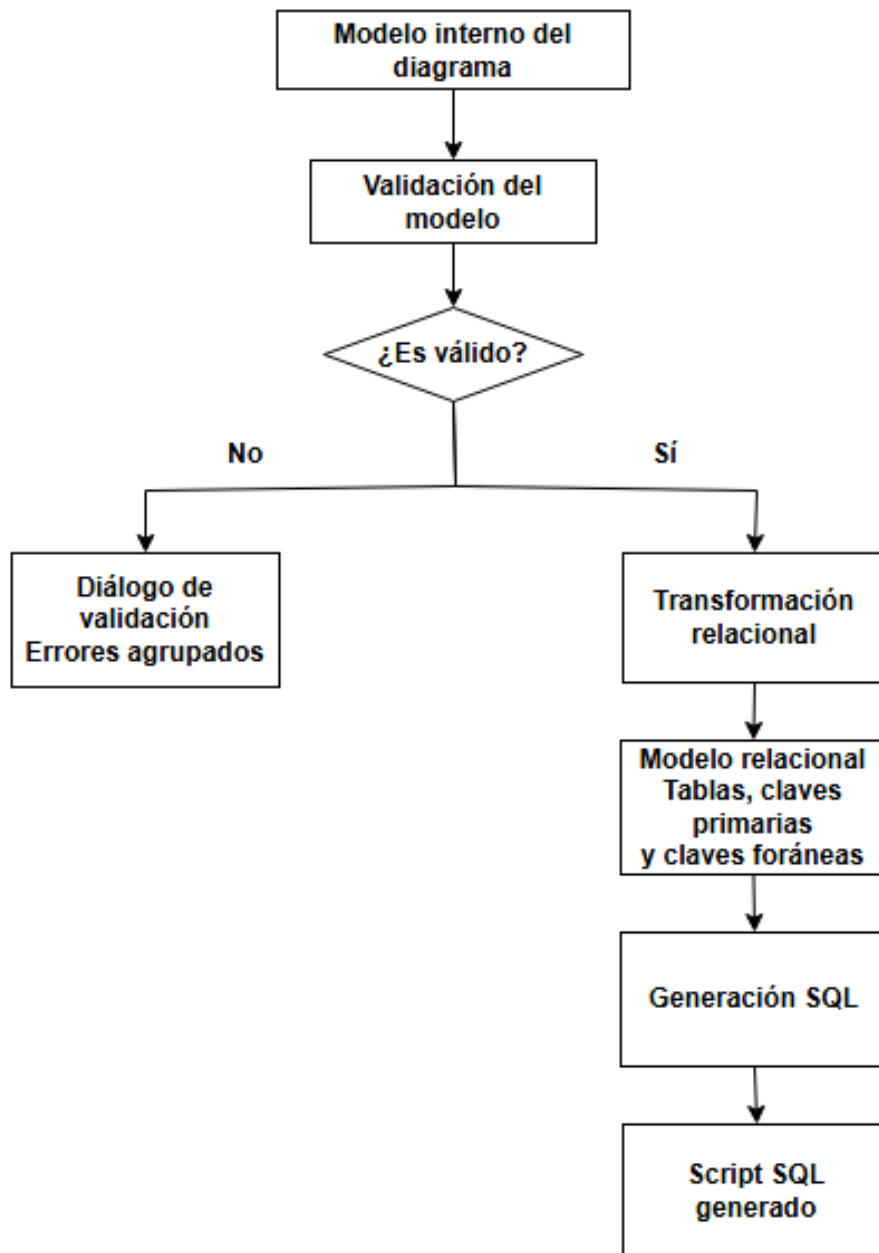


Figura C.5: Flujo de validación, transformación relacional y generación de SQL.

La Figura C.6 muestra el mismo proceso desde el punto de vista de la interacción entre los principales componentes implicados. La interfaz

del editor inicia la operación a partir de la acción del usuario, consulta la representación interna del diagrama, solicita la validación y, si no existen errores bloqueantes, continúa con la transformación relacional y la generación del script SQL.

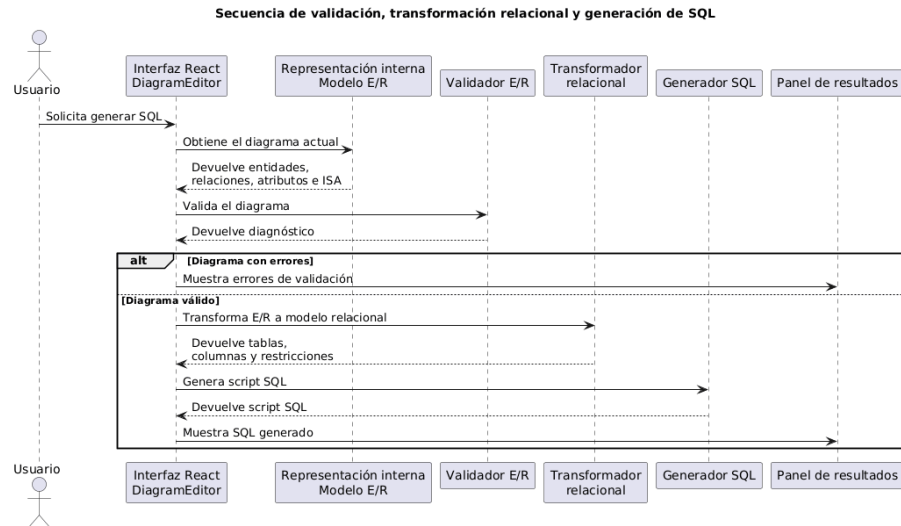


Figura C.6: Diagrama de secuencia de validación, transformación relacional y generación de SQL.

Importación, combinación y generación de estructuras

Además del flujo principal de edición, la aplicación incluye operaciones para importar diagramas JSON y generar estructuras básicas. En ambos casos, el usuario puede reemplazar el diagrama actual o combinar el nuevo contenido con el existente.

Cuando se combina contenido, la aplicación debe evitar conflictos directos de identificadores y mantener la coherencia de las referencias internas. Además, los elementos añadidos se insertan de forma controlada para facilitar su revisión visual dentro del lienzo.

Estas operaciones se diseñan como ayudas a la edición, no como mecanismos de fusión semántica avanzada. Por tanto, la aplicación no intenta interpretar si dos entidades representan el mismo concepto, sino insertar el contenido de forma controlada para que el usuario pueda revisarlo y ajustarlo posteriormente.

C.5. Diseño de interfaces

El diseño de la interfaz se centra en facilitar la edición de diagramas Entidad-Relación dentro de un lienzo web. La aplicación no se plantea como una herramienta de dibujo genérica, sino como un editor orientado a elementos concretos del modelo E-R. Por ello, las acciones disponibles dependen del tipo de elemento seleccionado y del contexto del diagrama.

La interfaz se organiza alrededor del lienzo principal, donde el usuario crea y manipula entidades, atributos, relaciones, entidades débiles, relaciones ternarias y jerarquías ISA sencillas. Junto al lienzo se ofrecen acciones generales, como validar el diagrama, generar SQL, importar o exportar JSON, exportar la imagen del diagrama, ajustar la vista, generar estructuras básicas y reiniciar el contenido previa confirmación.

Una decisión relevante del diseño de interfaz es el uso de acciones contextuales. Cuando el usuario selecciona un único elemento, la aplicación muestra acciones específicas para ese tipo de elemento, como añadir atributos, configurar relaciones, marcar una entidad como débil o trabajar con una jerarquía ISA. En cambio, cuando existe una selección múltiple, se ocultan las acciones individuales y se priorizan operaciones aplicables al conjunto, como mover o eliminar los elementos seleccionados.

También se incorporan elementos de apoyo al usuario, como la ayuda integrada, la información del proyecto y el cambio de idioma de la interfaz. Estas funcionalidades no modifican la semántica del modelo E-R, pero mejoran la comprensión del editor y facilitan su uso en un contexto académico.

El diseño visual mantiene la notación propia del modelo Entidad-Relación: entidades, relaciones, atributos, atributos clave, atributos compuestos, atributos multivaluados, entidades débiles, relaciones identificadoras e ISA se representan mediante convenciones gráficas diferenciadas. De esta forma, la interfaz busca que el usuario pueda reconocer los elementos del diagrama sin que las mejoras de usabilidad alteren su significado académico.

Apéndice *D*

Documentación técnica de programación

D.1. Introducción

Este anexo recoge la documentación técnica de programación de *UBU E-R App*. Su objetivo es servir como guía práctica para futuros desarrolladores que deban instalar el proyecto, ejecutarlo en local, comprender su estructura interna y continuar su evolución de forma controlada.

A diferencia del Anexo C, centrado en la especificación de diseño de la aplicación, este anexo tiene un enfoque operativo. Por ello, no se repiten en detalle los diagramas de arquitectura, el metamodelo del diagrama ni el proceso completo de transformación relacional. En su lugar, se describen los aspectos necesarios para trabajar con el código fuente: organización del repositorio, dependencias principales, scripts disponibles, ejecución local, construcción de la aplicación y pruebas.

El proyecto corresponde a la evolución de una aplicación web heredada para la edición de diagramas Entidad–Relación. La aplicación está desarrollada con React y utiliza `mxGraph` como librería principal para la representación y manipulación del lienzo visual. Durante el desarrollo del TFG se ha mantenido un criterio conservador: estabilizar y ampliar la herramienta sin reescribirla desde cero, preservando la coherencia entre el lienzo gráfico, la representación interna del diagrama, la validación, la transformación al modelo relacional, la generación de SQL y los mecanismos de persistencia.

D.2. Estructura de directorios

El repositorio del proyecto se organiza de forma convencional para una aplicación web basada en React. En la raíz del proyecto se encuentran los ficheros de configuración, la definición de dependencias, los scripts de ejecución y las carpetas principales de código, pruebas, documentación y recursos.

La estructura general más relevante es la siguiente:

```
/
+-- assets/
+-- docs/
+-- public/
+-- scripts/
+-- src/
+-- tests/
+-- package.json
+-- package-lock.json
+-- README.md
+-- LICENSE
+-- biome.json
+-- playwright.config.js
```

A continuación se describe la finalidad de los directorios y ficheros principales:

- **assets/**: contiene recursos gráficos generales del proyecto, como imágenes o animaciones utilizadas en la documentación del repositorio.
- **docs/**: contiene documentación auxiliar del proyecto. Dentro de esta carpeta se incluyen materiales de análisis, documentación $\text{\LaTeX}$ y recursos empleados en la memoria y anexos.
- **public/**: contiene los recursos estáticos servidos directamente por la aplicación React.
- **scripts/**: contiene scripts auxiliares de automatización. En particular, se utiliza para generar información de versión o construcción antes de arrancar o compilar la aplicación.

- **src/**: contiene el código fuente principal de la aplicación. Es la carpeta más relevante desde el punto de vista del desarrollo funcional del editor.
- **tests/**: contiene las pruebas automatizadas del proyecto, tanto pruebas unitarias como pruebas de extremo a extremo.
- **package.json**: define las dependencias del proyecto, sus metadatos y los scripts disponibles para instalación, ejecución, pruebas, análisis y construcción.
- **package-lock.json**: fija las versiones concretas de las dependencias instaladas, permitiendo reproducir el entorno de forma más estable.
- **README.md**: proporciona una descripción general del proyecto, sus funcionalidades principales, tecnología utilizada, instalación básica, pruebas y licencia.
- **LICENSE**: contiene la licencia del proyecto. La versión actual se distribuye bajo licencia MIT.
- **biome.json**: contiene la configuración de Biome, herramienta empleada para tareas de formato y análisis estático del código.
- **playwright.config.js**: contiene la configuración de Playwright para las pruebas de extremo a extremo.

La carpeta **src/** concentra la implementación de la aplicación. Su estructura actual separa el editor visual, las reglas del dominio E/R, la transformación relacional, la generación SQL y la internacionalización:

```
src/  
+-- components/  
| +-- DiagramEditor/  
+-- domain/  
| +-- er/  
| +-- relational/  
+-- i18n/  
+-- services/  
+-- App.js  
+-- buildInfo.js  
+-- index.js  
+-- styles.css
```

Los elementos principales de esta carpeta son:

- `index.js`: punto de entrada de la aplicación React.
- `App.js`: componente superior de la aplicación.
- `buildInfo.js`: fichero con información de construcción utilizada por la aplicación.
- `styles.css`: estilos globales compartidos por la interfaz.
- `components/DiagramEditor/`: contiene el editor visual de diagramas Entidad–Relación. Incluye el componente principal, hooks específicos, paneles de interfaz, estilos y utilidades de integración con `mxGraph`.
- `domain/er/`: contiene reglas y funciones auxiliares propias del dominio Entidad–Relación, como gestión de entidades, relaciones, atributos, jerarquías ISA, normalización y validación.
- `domain/relational/`: contiene la lógica de transformación desde la representación E/R hacia una representación relacional intermedia.
- `services/`: contiene servicios de generación y renderizado de SQL.
- `i18n/`: contiene el contexto de idioma y las cadenas de traducción utilizadas por la interfaz.

Dentro del editor principal, la estructura se ha reorganizado para reducir la concentración de responsabilidades en un único fichero:

```
src/components/DiagramEditor/  
+-- DiagramEditor.js  
+-- hooks/  
+-- panels/  
+-- styles/  
+-- utils/
```

El fichero `DiagramEditor.js` actúa como componente orquestador del editor. Mantiene las referencias principales del lienzo, coordina la inicialización de `mxGraph`, compone los hooks y paneles, y conecta la interacción visual con la representación interna del diagrama.

La carpeta `hooks/` agrupa lógica de acciones y servicios asociados al editor:

```
hooks/  
+-- useAttributeActions.js  
+-- useDeletionActions.js  
+-- useDiagramHistory.js  
+-- useDiagramPersistence.js  
+-- useIsaActions.js  
+-- useRelationActions.js
```

Estos hooks encapsulan responsabilidades como la gestión del historial, la persistencia, las acciones sobre atributos, relaciones, jerarquías ISA y operaciones de borrado.

La carpeta `panels/` contiene componentes de interfaz del panel lateral:

```
panels/  
+-- DiagramEditorGlobalActions.js  
+-- DiagramEditorPanelControls.js  
+-- DiagramEditorSidebar.js
```

En ella se separan las acciones generales del editor, los controles compartidos y las acciones contextuales dependientes de la selección actual.

Por último, la carpeta `utils/` contiene utilidades agrupadas por responsabilidad técnica:

```
utils/  
+-- graph/  
+-- mxStyles/  
+-- persistence/  
+-- rendering/  
+-- selection/  
+-- sync/  
+-- toolbar/  
+-- validation/
```

Estas utilidades incluyen la integración con `mxGraph`, los estilos visuales de las celdas, la importación y exportación de diagramas, el renderizado de elementos, la selección, la sincronización entre modelo y grafo, la barra de herramientas y los mensajes de validación mostrados al usuario.

Esta organización busca facilitar el mantenimiento del proyecto sin ocultar el carácter heredado de la aplicación. Algunas partes, especialmente la inicialización de `mxGraph` y determinados efectos ligados al ciclo de vida del editor, permanecen en el componente principal por prudencia, ya que están fuertemente acopladas al funcionamiento del lienzo visual.

D.3. Manual del programador

Esta sección describe los pasos necesarios para preparar el entorno de desarrollo, instalar las dependencias, ejecutar la aplicación en local y utilizar los scripts principales del proyecto. El objetivo no es documentar cada fichero de la aplicación, sino proporcionar una guía suficiente para que otro desarrollador pueda continuar el trabajo sin depender únicamente del conocimiento previo del autor.

Requisitos previos

Para trabajar con el proyecto es necesario disponer de un entorno de desarrollo web basado en Node.js y npm. La aplicación se ejecuta completamente en el navegador y no requiere la instalación de un servidor de bases de datos ni de un backend propio.

Los requisitos principales son:

- **Node.js:** entorno de ejecución necesario para instalar dependencias, arrancar la aplicación y generar la versión de producción.
- **npm:** gestor de paquetes utilizado para instalar las dependencias declaradas en el fichero `package.json` y ejecutar los scripts del proyecto.
- **Navegador web moderno:** necesario para utilizar la aplicación durante el desarrollo. Se recomienda emplear un navegador actualizado, especialmente al ejecutar pruebas manuales sobre el lienzo.
- **Editor de código:** se puede utilizar cualquier editor compatible con proyectos JavaScript. Durante el desarrollo se ha trabajado principalmente con Visual Studio Code.

El proyecto incluye un fichero `.node-version`, que sirve como referencia de la versión de Node.js utilizada en el entorno de desarrollo. Es recomendable respetar dicha versión o, al menos, emplear una versión compatible con `react-scripts` y con las dependencias instaladas.

Instalación de dependencias

Una vez descargado o clonado el repositorio, la instalación de dependencias se realiza desde la raíz del proyecto mediante el siguiente comando:

```
npm install
```

Este comando lee el fichero `package.json` y utiliza `package-lock.json` para instalar versiones concretas de las dependencias. Es importante conservar el fichero `package-lock.json`, ya que ayuda a reproducir el entorno de instalación y reduce diferencias entre equipos.

Las dependencias principales de ejecución incluyen React, mxGraph, Material UI, fuentes de Roboto, utilidades de notificación y librerías auxiliares de interfaz. Las dependencias de desarrollo incluyen herramientas de pruebas, cobertura, análisis estático y automatización.

Ejecución en entorno de desarrollo

Para arrancar la aplicación en local se utiliza el script:

```
npm start
```

Antes de ejecutar el servidor de desarrollo, el proyecto lanza automáticamente el script `prestart`, encargado de generar información de construcción mediante `scripts/generateBuildInfo.js`. Después, `react-scripts` inicia la aplicación en modo desarrollo.

Una vez arrancada, la aplicación queda disponible en el navegador en la dirección local indicada por la terminal, normalmente:

```
http://localhost:3000
```

Durante esta ejecución se pueden realizar pruebas manuales del editor: creación de entidades, relaciones, atributos, jerarquías ISA, validación del diagrama, generación de SQL, importación y exportación de diagramas y comprobación de la persistencia local.

Construcción de la aplicación

Para generar una versión de producción se utiliza el comando:

```
npm run build
```

De forma análoga al arranque en desarrollo, antes de construir la aplicación se ejecuta el script `prebuild`, que actualiza la información de construcción. Posteriormente, `react-scripts` genera una versión optimizada de la aplicación en la carpeta `build/`.

La carpeta `build/` contiene los ficheros estáticos resultantes, preparados para ser servidos por una plataforma de despliegue web. En el proyecto actual, el despliegue se realiza sobre Vercel, que se encarga de construir y publicar la aplicación a partir del repositorio.

La aplicación no depende de un backend propio. La lógica principal se ejecuta en el navegador del usuario y la persistencia ordinaria del diagrama se realiza mediante `localStorage` y mediante ficheros JSON importados o exportados por el usuario.

Scripts disponibles

Los scripts principales del proyecto están definidos en el fichero `package.json`. Los más relevantes para un desarrollador son los siguientes:

- `npm start`: arranca la aplicación en modo desarrollo.
- `npm run build`: genera la versión de producción de la aplicación.
- `npm test`: ejecuta la batería de pruebas unitarias con Vitest, excluyendo las pruebas de extremo a extremo.
- `npm test - --run`: ejecuta las pruebas unitarias en modo no interactivo, adecuado para una comprobación puntual antes de cerrar cambios.
- `npm run test:e2e`: ejecuta las pruebas de extremo a extremo con Playwright.
- `npm run test:coverage`: ejecuta las pruebas unitarias y genera información de cobertura.
- `npm run lint`: ejecuta Biome sobre la carpeta `src/` para detectar y corregir problemas de análisis estático cuando sea posible.

- `npm run format`: aplica el formateo de Biome sobre la carpeta `src/`.

En el trabajo diario se recomienda ejecutar, como mínimo, las pruebas unitarias y el análisis estático antes de considerar cerrado un cambio. Para modificaciones que afecten al comportamiento visual del editor, a la interacción con mxGraph, a la persistencia o a flujos completos de usuario, también conviene ejecutar las pruebas de extremo a extremo o realizar una revisión manual específica en el navegador.

Consideraciones sobre el entorno

El proyecto procede de una base heredada y utiliza `react-scripts`, por lo que pueden aparecer advertencias asociadas a dependencias internas o herramientas de construcción. Durante la revisión final del código se comprobó que dichas advertencias no bloqueaban la ejecución, las pruebas ni la generación de la build.

Si aparecen errores al instalar o ejecutar el proyecto, se recomienda revisar primero:

- la versión de Node.js utilizada;
- que las dependencias se hayan instalado desde la raíz del proyecto;
- que no existan restos inconsistentes en `node_modules/`;
- que se esté usando el fichero `package-lock.json` incluido en el repositorio;
- que Playwright tenga instalados los navegadores necesarios si se van a ejecutar pruebas de extremo a extremo.

En caso de problemas persistentes de dependencias, una estrategia habitual consiste en eliminar la carpeta de dependencias instaladas y reinstalar el proyecto mediante `npm install`. Esta operación debe realizarse con prudencia, evitando modificar manualmente el fichero `package-lock.json` salvo que se esté actualizando una dependencia de forma justificada.

D.4. Organización interna del código fuente

La aplicación se organiza en torno a una separación progresiva entre la interfaz React, la integración con mxGraph, las reglas propias del modelo Entidad-Relación, la transformación al modelo relacional y la generación de SQL. Esta separación no elimina por completo el acoplamiento propio de una aplicación heredada, pero permite localizar mejor las responsabilidades principales del sistema.

Componente principal del editor

El componente principal del editor se encuentra en:

```
src/components/DiagramEditor/DiagramEditor.js
```

Este fichero actúa como orquestador del editor. Su responsabilidad principal es coordinar la inicialización del lienzo, las referencias internas, los efectos de ciclo de vida, los hooks de acciones y los paneles de interfaz.

Durante el desarrollo del TFG se redujo progresivamente la concentración de responsabilidades de este componente. No obstante, no se realizó una reescritura completa ni se extrajo toda la inicialización de mxGraph, ya que esta parte del sistema está fuertemente acoplada a la creación del grafo, la gestión de eventos, la sincronización geométrica y la reconstrucción visual del diagrama.

Por este motivo, `DiagramEditor.js` conserva las responsabilidades de coordinación más sensibles, mientras que parte de la lógica de acciones y de interfaz se ha movido a hooks y componentes auxiliares.

Hooks del editor

La carpeta de hooks del editor agrupa lógica asociada a acciones concretas del usuario y servicios internos del componente:

```
src/components/DiagramEditor/hooks/
```

Los hooks principales son:

- `useDiagramHistory.js`: gestiona el historial de cambios del diagrama, permitiendo operaciones de deshacer y rehacer.

- `useDiagramPersistence.js`: agrupa operaciones relacionadas con persistencia, exportación, importación y restauración del diagrama.
- `useAttributeActions.js`: contiene acciones relacionadas con atributos, como creación, modificación y operaciones sobre atributos compuestos o multivaluados.
- `useRelationActions.js`: contiene acciones relacionadas con relaciones, participantes, cardinalidades, roles y relaciones identificadoras.
- `useIsaActions.js`: agrupa acciones relacionadas con la creación, configuración y actualización de jerarquías ISA.
- `useDeletionActions.js`: centraliza operaciones de borrado de elementos del diagrama, incluyendo casos con dependencias visuales o internas.

Esta división facilita el mantenimiento del editor, ya que evita que todas las operaciones de usuario se concentren en el componente principal. Aun así, los hooks siguen dependiendo de referencias compartidas del editor, por lo que cualquier modificación debe hacerse con cuidado para no romper la sincronización entre el grafo visual y la representación interna.

Paneles de interfaz

La interfaz lateral del editor se divide en componentes ubicados en:

`src/components/DiagramEditor/panels/`

Los ficheros principales son:

- `DiagramEditorGlobalActions.js`: contiene las acciones generales del editor, como creación de elementos, historial, validación, generación SQL, importación, exportación y ayuda.
- `DiagramEditorSidebar.js`: contiene las acciones contextuales que dependen del elemento seleccionado en el lienzo.
- `DiagramEditorPanelControls.js`: agrupa controles compartidos utilizados por los paneles.

Esta separación permite distinguir entre acciones globales, disponibles con independencia de la selección, y acciones contextuales, que solo tienen sentido cuando el usuario ha seleccionado una entidad, relación, atributo o jerarquía ISA.

Utilidades de integración con mxGraph

La carpeta de utilidades del editor contiene funciones auxiliares ligadas a la representación visual, la interacción con mxGraph, la reconstrucción del diagrama y la sincronización con la representación interna:

`src/components/DiagramEditor/utils/`

Dentro de esta carpeta destacan los siguientes grupos:

- **graph/**: configuración e interacción básica con mxGraph, incluyendo inicialización, comportamiento del grafo y edición de etiquetas.
- **mxStyles/**: estilos visuales utilizados para representar entidades, relaciones, atributos, cardinalidades y otros elementos del diagrama.
- **rendering/**: funciones encargadas de construir o actualizar la representación visual de entidades, relaciones, atributos y decoradores.
- **sync/**: funciones relacionadas con la sincronización entre la representación interna del diagrama y el grafo visual de mxGraph.
- **persistence/**: funciones de importación y exportación de diagramas mediante ficheros.
- **selection/**: utilidades relacionadas con la selección de elementos del lienzo.
- **toolbar/**: funciones auxiliares para la creación de herramientas del editor.
- **validation/**: mensajes de validación que se muestran al usuario a partir de los diagnósticos internos.

La parte más delicada de esta zona es la sincronización entre modelo y grafo. La aplicación mantiene una representación interna del diagrama y, al mismo tiempo, una representación visual gestionada por mxGraph. Por tanto, cualquier cambio que afecte a identificadores, estilos, geometría, celdas o reconstrucción visual debe revisarse cuidadosamente.

Dominio Entidad–Relación

Las reglas y utilidades propias del dominio Entidad–Relación se encuentran en:

```
src/domain/er/
```

Esta carpeta contiene funciones relacionadas con entidades, relaciones, atributos, composición del diagrama, normalización, ejemplos predefinidos, jerarquías ISA y validación.

Entre sus ficheros principales se encuentran:

- `entities.js`: funciones auxiliares relacionadas con entidades del diagrama.
- `relations.js`: funciones auxiliares relacionadas con relaciones y participantes.
- `attributes.js`: funciones de apoyo para atributos simples, compuestos, multivaluados, claves y discriminantes.
- `isa.js`: funciones de dominio relacionadas con jerarquías ISA.
- `diagramComposition.js`: utilidades para componer o consultar elementos del diagrama.
- `diagramNormalization.js`: funciones de normalización de la representación interna.
- `examples.js`: generación de estructuras o ejemplos predefinidos.
- `validation/`: reglas de validación del diagrama E/R.

La validación se organiza mediante reglas específicas para entidades, relaciones, atributos, entidades débiles, ISA e identificadores SQL. Esta división permite añadir nuevas comprobaciones de forma más localizada, evitando concentrar todas las reglas en una única función.

Transformación al modelo relacional

La transformación desde el modelo Entidad–Relación hacia una representación relacional intermedia se encuentra en:

`src/domain/relational/`

El fichero principal es:

`src/domain/relational/erToRelationalModel.js`

Este módulo construye una representación relacional a partir del diagrama validado. Para ello se apoya en funciones auxiliares de proyección de atributos, generación de columnas de clave, normalización de nombres y tratamiento de entidades, relaciones, entidades débiles, relaciones ternarias y jerarquías ISA.

La existencia de una representación relacional intermedia evita que la generación SQL dependa directamente de todos los detalles visuales del diagrama. De esta forma, el proceso queda dividido en dos pasos: primero se transforma el diagrama E/R a una estructura relacional y, después, esa estructura se renderiza como SQL.

Generación de SQL

La generación de SQL se encuentra en la carpeta de servicios:

`src/services/`

Los ficheros principales son:

- `sql.js`: coordina la generación del SQL a partir del modelo relacional.
- `sqlRenderer.js`: contiene funciones específicas para renderizar tablas, columnas, claves primarias y claves foráneas en formato SQL.

El SQL generado está orientado a PostgreSQL y debe interpretarse como una salida simplificada de apoyo académico. Su objetivo es representar de forma razonable las decisiones principales del diagrama E/R, no sustituir a una herramienta industrial completa de diseño físico de bases de datos.

Internacionalización

La aplicación incluye un selector de idioma y centraliza las cadenas de texto en:

`src/i18n/`

Los ficheros principales son:

- `LanguageContext.js`: define el contexto de idioma utilizado por la interfaz.
- `translations.js`: contiene las cadenas traducidas que se muestran al usuario.

Cuando se añadan nuevas acciones, mensajes de validación o textos de interfaz, debe comprobarse si procede incorporarlos a este sistema de traducción en lugar de dejarlos como literales dispersos en los componentes.

D.5. Pruebas del sistema

Las pruebas del sistema combinan comprobaciones automáticas y revisiones manuales. Esta combinación es necesaria porque la aplicación tiene una parte importante de lógica de dominio, que puede verificarse mediante pruebas unitarias, y una parte visual e interactiva basada en `mxGraph`, que requiere comprobar también el comportamiento real del editor en el navegador.

Durante el desarrollo se utilizaron principalmente cuatro tipos de comprobación:

- análisis estático del código;
- pruebas unitarias;
- pruebas de extremo a extremo;
- construcción de la aplicación para producción.

Además, para los cambios que afectaban al comportamiento visual del editor se realizaron revisiones manuales, especialmente en operaciones relacionadas con selección, movimiento de elementos, reconstrucción del diagrama, importación, exportación y generación visual de estructuras.

Análisis estático

El análisis estático se realiza mediante Biome. Esta herramienta permite detectar problemas de estilo, código no utilizado, errores simples y otras inconsistencias en el código fuente.

El comando utilizado es:

```
npm run lint
```

Este comando se aplica sobre la carpeta `src/`. Se recomienda ejecutarlo antes de cerrar cualquier cambio relevante, especialmente si se han modificado componentes React, hooks, utilidades del editor o funciones del dominio E/R.

Durante la limpieza final del código fuente, este comando se utilizó para comprobar que no quedaban errores de análisis estático tras eliminar código muerto, imports no utilizados y helpers redundantes.

Pruebas unitarias

Las pruebas unitarias se ejecutan con Vitest. Estas pruebas se centran principalmente en la lógica interna de la aplicación, como validación del diagrama, transformación al modelo relacional, generación SQL, persistencia, normalización de datos y funciones auxiliares del dominio.

El comando recomendado para una ejecución puntual es:

```
npm test
```

Durante el desarrollo también puede utilizarse la variante siguiente cuando se quiera forzar una ejecución no interactiva:

```
npm test -- --run
```

```

PS C:\Users\spaa_\Documents\TFG\code\draw-entity-relation> npm test
DEV v2.0.0 C:\Users\spaa_\Documents\TFG\code\draw-entity-relation

✓ tests/unit/editor/attributeRendering.test.js (12)
✓ tests/unit/editor/attributeSelection.test.js (7)
✓ tests/unit/editor/decoratorRendering.test.js (5)
✓ tests/unit/editor/diagramGraphSync.test.js (2)
✓ tests/unit/editor/diagramReconstruction.test.js (3)
✓ tests/unit/editor/isaGraphCanvas.test.js (1)
✓ tests/unit/er/attributes.test.js (31)
✓ tests/unit/er/diagramComposition.test.js (7)
✓ tests/unit/er/diagramNormalization.test.js (11)
✓ tests/unit/er/isa.test.js (12)
✓ tests/unit/er/relations.test.js (17)
✓ tests/unit/relational/attributeProjection.test.js (7)
✓ tests/unit/relational/binaryRelations.test.js (8)
✓ tests/unit/relational/entityKeyColumns.test.js (4)
✓ tests/unit/relational/isaRelations.test.js (2)
✓ tests/unit/relational/nmRelations.test.js (5)
✓ tests/unit/relational/tableFiltering.test.js (4)
✓ tests/unit/relational/ternaryRelations.test.js (11)
✓ tests/unit/sql/entities.test.js (7)
✓ tests/unit/sql/general.test.js (8)
✓ tests/unit/sql/isa.test.js (4)
✓ tests/unit/sql/nm-relations.test.js (1)
✓ tests/unit/sql/relations.test.js (5)
✓ tests/unit/sql/ternaryRelations.test.js (4)
✓ tests/unit/sql/weak-entities.test.js (9)
✓ tests/unit/validation/attributes.test.js (21)
✓ tests/unit/validation/entities.test.js (10)
✓ tests/unit/validation/general.test.js (2)
✓ tests/unit/validation/isa.test.js (9)
✓ tests/unit/validation/relations.test.js (19)
✓ tests/unit/validation/sql-identifiers.test.js (9)
✓ tests/unit/validation/weak-entities.test.js (28)

Test Files 32 passed (32)
Tests 285 passed (285)
Start at 11:00:07
Duration 3.78s (transform 729ms, setup 4ms, collect 3.24s, tests 512ms, environment 8ms, prepare 6.58s)

```

Figura D.1: Ejecución de las pruebas unitarias del proyecto mediante Vitest.

En la comprobación final realizada tras la limpieza y reorganización del código fuente, la batería de pruebas unitarias se ejecutó correctamente, con 32 ficheros de test y 285 pruebas superadas. Este resultado permite considerar que los cambios aplicados no rompieron las comprobaciones automáticas existentes.

No obstante, conviene interpretar estas pruebas con prudencia. Que una funcionalidad esté cubierta por pruebas unitarias no significa que todos sus casos de uso visuales estén completamente verificados, especialmente en las partes dependientes de mxGraph y de la interacción directa del usuario con el lienzo.

Pruebas de extremo a extremo

El proyecto también dispone de pruebas de extremo a extremo mediante Playwright. Estas pruebas permiten verificar flujos completos de uso desde

el punto de vista del usuario, ejecutando la aplicación en un navegador controlado por la herramienta.

El comando definido para este tipo de pruebas es:

```
npm run test:e2e
```

```
PS C:\Users\spaa_\Documents\TFG\code\draw-entity-relation> npm run test:e2e

> ubu-er-app@1.0.0 test:e2e
> npx playwright test
[WebServer] (node:22668) [DEP0176] DeprecationWarning: fs.F_OK is deprecated, use fs.access instead
(Use `node --trace-deprecation ...` to show where the warning was created)
[WebServer] (node:22668) [DEP_WEBPACK_DEV_SERVER_ON_AFTER_SETUP_MIDDLEWARE] DeprecationWarning: onAfterSetupMiddleware is deprecated.
(node:22668) [DEP_WEBPACK_DEV_SERVER_ON_BEFORE_SETUP_MIDDLEWARE] DeprecationWarning: onBeforeSetupMiddleware is deprecated.

Running 148 tests using 6 workers
[WebServer] One of your dependencies, babel-preset-react-app, is importing the
"@babel/plugin-proposal-private-property-in-object" package without
declaring it in its dependencies. This is currently working because
"@babel/plugin-proposal-private-property-in-object" is already in your
node_modules folder for unrelated reasons, but it may break at any time.

babel-preset-react-app is part of the create-react-app project, which
is not maintained anymore. It is thus unlikely that this bug will
ever be fixed. Add "@babel/plugin-proposal-private-property-in-object" to
your devDependencies to work around this error. This will make this message
go away.

148 passed (1.7m)

To open last HTML report run:

npx playwright show-report
```

Figura D.2: Ejecución de las pruebas de extremo a extremo mediante Playwright.

Estas pruebas son especialmente útiles para comprobar escenarios en los que intervienen varios elementos de la aplicación: carga del editor, interacción con la interfaz, creación de elementos, validación de acciones y comportamiento general del sistema en navegador. En el contexto de este proyecto complementan a las pruebas unitarias, ya que permiten detectar regresiones en flujos completos que dependen de la interfaz y no solo de funciones aisladas.

En la ejecución mostrada se completaron correctamente 148 pruebas de extremo a extremo. Durante la ejecución pueden aparecer advertencias heredadas asociadas a **react-scripts** y a dependencias internas de Babel. Estas advertencias no impiden la ejecución de las pruebas ni suponen, por sí mismas, un fallo funcional de la aplicación.

Cobertura de pruebas

El proyecto incluye un script para obtener información de cobertura de las pruebas unitarias:

```
npm run test:coverage
```

Este comando permite identificar qué partes del código están siendo ejercitadas por las pruebas automáticas. La cobertura debe utilizarse como una métrica de apoyo, no como único criterio de calidad. En una aplicación de estas características, una cobertura elevada puede no reflejar por completo la corrección de la interacción visual con el lienzo ni la usabilidad de los flujos de edición.

Por ello, la cobertura resulta útil para detectar zonas de lógica interna con poca validación automática, pero debe complementarse con revisión manual y pruebas de integración cuando se modifiquen comportamientos complejos.

Construcción de producción

La construcción de producción permite comprobar que la aplicación puede compilarse correctamente y generar los ficheros estáticos necesarios para su despliegue.

El comando utilizado es:

```
npm run build
```

Este comando ejecuta previamente el script de generación de información de construcción y, después, lanza el proceso de **react-scripts build**. Si la compilación finaliza correctamente, se genera la carpeta **build/**, que contiene la versión optimizada de la aplicación.

Durante la comprobación final, la build se generó correctamente. Únicamente aparecieron advertencias heredadas asociadas a **react-scripts** y

Babel, que no impidieron la generación de la aplicación ni bloquearon su ejecución.

Revisión manual

Además de las pruebas automáticas, se realizaron revisiones manuales del comportamiento de la aplicación en el navegador. Estas revisiones son importantes porque parte del sistema depende de la coordinación entre React, mxGraph, la representación interna del diagrama y la interacción directa del usuario.

Entre los aspectos revisados manualmente destacan:

- creación y edición de entidades, relaciones y atributos;
- creación y configuración de jerarquías ISA;
- selección simple y múltiple de elementos;
- movimiento y borrado de varios elementos;
- validación del diagrama;
- transformación al modelo relacional;
- generación de SQL compatible con PostgreSQL;
- persistencia local;
- importación y exportación JSON;
- reconstrucción visual del diagrama importado;
- exportación visual del lienzo;
- ajuste de vista y uso del panel lateral.

Estas comprobaciones no sustituyen a las pruebas automáticas, pero permiten detectar regresiones visuales o de interacción que no siempre aparecen en pruebas unitarias. Un ejemplo de este tipo de revisión fue la detección de un problema de orden en el panel lateral, donde las secciones de selección y orden aparecían al final del panel en lugar de mostrarse junto a las herramientas principales.

Comprobación recomendada antes de cerrar cambios

Antes de considerar cerrado un cambio relevante en el código fuente, se recomienda ejecutar al menos la siguiente secuencia:

```
npm run lint
npm test -- --run
npm run build
```

Si el cambio afecta a flujos completos de usuario, interacción visual, importación, exportación, persistencia o reconstrucción del lienzo, también se recomienda ejecutar:

```
npm run test:e2e
```

y realizar una revisión manual en el navegador.

Esta pauta no garantiza por sí sola la ausencia completa de errores, pero reduce el riesgo de introducir regresiones en una aplicación heredada con acoplamiento entre modelo interno, representación visual y lógica de transformación.

La figura siguiente muestra una comprobación de análisis estático y construcción de producción. La ejecución de pruebas unitarias se documenta previamente en la Figura [D.1](#).

```
● PS C:\Users\spaa_\Documents\TFG\code\draw-entity-relation> npm run lint

> ubu-er-app@1.0.0 lint
> npx @biomejs/biome lint --apply ./src

Checked 56 files in 43ms. No fixes needed.
● PS C:\Users\spaa_\Documents\TFG\code\draw-entity-relation> npm run build

> ubu-er-app@1.0.0 prebuild
> node scripts/generateBuildInfo.js

[build-info] v1.0.0 · 28/06/2026 · ea14fa7

> ubu-er-app@1.0.0 build
> react-scripts build

(node:26592) [DEP0176] DeprecationWarning: fs.F_OK is deprecated, use fs.constants.F_OK instead.
(Use `node --trace-deprecation ...` to show where the warning was created)
Creating an optimized production build...
One of your dependencies, babel-preset-react-app, is importing the
"@babel/plugin-proposal-private-property-in-object" package without
declaring it in its dependencies. This is currently working because
"@babel/plugin-proposal-private-property-in-object" is already in your
node_modules folder for unrelated reasons, but it may break at any time.

babel-preset-react-app is part of the create-react-app project, which
is not maintained anymore. It is thus unlikely that this bug will
ever be fixed. Add "@babel/plugin-proposal-private-property-in-object" to
your devDependencies to work around this error. This will make this message
go away.

Compiled successfully.
```

Figura D.3: Comprobación de análisis estático y construcción de producción del proyecto.

Documentación de usuario

E.1. Introducción

Este anexo presenta una guía de uso de UBU E-R App desde el punto de vista del usuario final. Su objetivo es explicar cómo acceder a la aplicación y cómo utilizar sus principales funcionalidades para crear, validar, importar, exportar y transformar diagramas Entidad-Relación.

A diferencia de la documentación técnica de programación, este apartado no describe la estructura interna del código ni las decisiones de implementación. El propósito es ofrecer una referencia práctica para manejar la aplicación de forma autónoma, explicando las acciones habituales sobre el lienzo y sobre los elementos del diagrama.

Para facilitar el seguimiento del manual, las acciones se explican mediante un ejemplo sencillo basado en empleados y oficinas. Este ejemplo permite mostrar de forma progresiva la creación de entidades, atributos, claves primarias, relaciones, cardinalidades, validación del diagrama y generación de SQL. El manual no pretende cubrir todas las combinaciones posibles del modelo E/R, sino presentar un flujo representativo de uso de la aplicación.

La aplicación está orientada principalmente a usuarios con conocimientos básicos del modelo Entidad-Relación. Por ello, se asume que el usuario conoce los conceptos generales de entidad, atributo, relación, cardinalidad y clave primaria, aunque la propia interfaz incorpora mensajes de ayuda, acciones contextuales y validaciones para facilitar el trabajo con el editor.

E.2. Requisitos de usuario

Para utilizar la aplicación, el usuario debe cumplir los siguientes requisitos básicos:

- Disponer de un navegador web actualizado. Durante el desarrollo se ha trabajado y probado principalmente con navegadores basados en Chromium y con Firefox.
- Tener conexión a Internet si se utiliza la versión desplegada de la aplicación.
- Contar con conocimientos básicos sobre diagramas Entidad-Relación y sobre su transformación al modelo relacional.
- Disponer de un dispositivo con pantalla y ratón, panel táctil o mecanismo equivalente que permita interactuar con el lienzo gráfico.

Aunque la aplicación ofrece ayuda contextual y mensajes de validación, es recomendable que el usuario conozca previamente los elementos principales del modelo E/R. En particular, resulta útil entender la diferencia entre entidades fuertes y débiles, atributos simples, compuestos o multivaluados, relaciones binarias y ternarias, cardinalidades y claves primarias.

E.3. Instalación

La forma principal de utilizar UBU E-R App es mediante la versión desplegada de la aplicación. En este caso no es necesario instalar nada en el equipo del usuario, ya que la aplicación se ejecuta directamente desde el navegador web:

Aplicación web desplegada

Para utilizar esta versión basta con abrir el enlace anterior en un navegador actualizado. La aplicación no requiere autenticación, registro de usuario ni configuración previa. El trabajo realizado se conserva en el almacenamiento local del navegador y también puede guardarse mediante la exportación del diagrama en formato JSON.

De forma alternativa, el proyecto puede ejecutarse localmente. Esta opción está orientada principalmente a revisión técnica, pruebas o continuación

del desarrollo, por lo que se describe con más detalle en el manual del programador. De manera resumida, se requiere tener instalados Node.js y npm, clonar el repositorio, instalar las dependencias y arrancar el servidor de desarrollo:

```
git clone <url-del-repositorio>
cd <directorio-del-proyecto>
npm install
npm start
```

Tras ejecutar estos comandos, la aplicación estará disponible normalmente en la dirección local indicada por la terminal, habitualmente:

`http://localhost:3000`

También puede generarse una versión preparada para producción mediante:

```
npm run build
```

Esta ejecución local no es necesaria para el uso habitual de la aplicación, pero permite revisar el funcionamiento del proyecto fuera del despliegue web.

E.4. Manual del usuario

En esta sección se describe el uso principal de la aplicación mediante un recorrido guiado. El ejemplo utilizado representa una situación sencilla en la que una organización tiene empleados que trabajan en oficinas. A partir de este caso se muestran las operaciones habituales del editor: crear entidades, añadir atributos, configurar relaciones, validar el diagrama, generar SQL y guardar o exportar el trabajo.

Pantalla principal

Al acceder a la aplicación se muestra la pantalla principal del editor. La interfaz está formada por un panel lateral de herramientas y acciones, y un lienzo central donde se construye el diagrama Entidad-Relación.

En la figura se muestra la pantalla principal con las dos entidades iniciales del ejemplo utilizado en el manual.



Figura E.1: Pantalla principal de UBU E-R App con las entidades iniciales del ejemplo.

La zona principal de trabajo es el lienzo. Sobre él se colocan las entidades, atributos, relaciones y otros elementos del modelo. El usuario puede seleccionar los elementos creados para moverlos, modificarlos o aplicar acciones contextuales.

En el panel lateral se encuentran las herramientas de creación, las acciones contextuales y las acciones generales del diagrama. Estas opciones permiten crear nuevos elementos, modificar los elementos seleccionados, validar el modelo, generar SQL, importar o exportar diagramas y acceder a la ayuda de la aplicación.

Cuando no hay ningún elemento seleccionado, la aplicación muestra las acciones generales disponibles. Al seleccionar un elemento del lienzo, el panel lateral se adapta y muestra las acciones que tienen sentido para ese tipo de elemento.

Componentes de la interfaz

La interfaz de la aplicación se organiza en dos zonas principales. Por un lado, el panel lateral agrupa el acceso al cambio de idioma, las herramientas de creación, las acciones contextuales y las acciones generales del diagrama. Por otro lado, el lienzo de edición ocupa la zona central y permite construir visualmente el modelo Entidad-Relación.

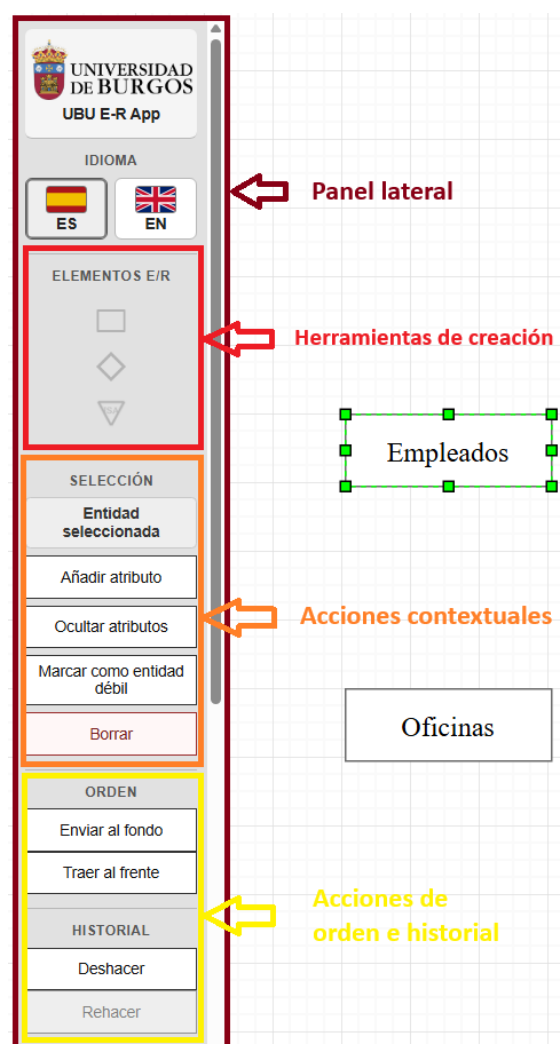


Figura E.2: Panel lateral con herramientas de creación y acciones contextuales de una entidad seleccionada.

Al seleccionar un elemento del lienzo, el panel lateral muestra acciones contextuales asociadas a ese tipo de elemento. En la figura anterior se

muestra el caso de una entidad seleccionada, donde aparecen operaciones como añadir atributos, ocultar atributos, marcar la entidad como débil, modificar el orden visual o eliminarla.

Además de las acciones contextuales, la aplicación incorpora acciones generales que afectan al diagrama completo. Estas acciones permiten generar estructuras de ejemplo, comprobar el diagrama, ajustar la vista, generar SQL, exportar o importar diagramas, exportar una imagen del lienzo y reiniciar el trabajo actual.



Figura E.3: Panel lateral con acciones generales, importación, exportación y ayuda del editor.

Esta separación entre acciones contextuales y acciones generales permite que el usuario distinga entre operaciones aplicadas a un elemento concreto y operaciones aplicadas al conjunto del diagrama.

Crear un diagrama sencillo

El ejemplo del manual se basa en dos entidades: *Empleados* y *Oficinas*. La entidad *Empleados* representa a los trabajadores de una organización, mientras que *Oficinas* representa las sedes en las que pueden trabajar.

Para comenzar el diagrama se siguen estos pasos:

1. Arrastrar una entidad desde el panel lateral hasta el lienzo.
2. Cambiar el nombre de la entidad a *Empleados*.
3. Arrastrar una segunda entidad al lienzo.
4. Cambiar el nombre de la segunda entidad a *Oficinas*.
5. Colocar ambas entidades dejando espacio entre ellas para añadir posteriormente la relación.

Una vez creadas las entidades, se añaden sus atributos principales. Para la entidad *Empleados* se pueden definir los atributos *idEmpleado*, *nombre* y *email*. Para la entidad *Oficinas* se pueden definir los atributos *idOficina*, *nombre* y *ciudad*.

Para añadir atributos a una entidad:

1. Seleccionar la entidad en el lienzo.
2. Utilizar la acción contextual para añadir un atributo.
3. Introducir el nombre del atributo.
4. Repetir el proceso hasta completar los atributos necesarios.

Después de añadir los atributos, deben marcarse las claves primarias. En este ejemplo, *idEmpleado* identifica a cada empleado e *idOficina* identifica a cada oficina.

Para marcar un atributo como clave primaria:

1. Seleccionar el atributo correspondiente.
2. Utilizar la acción contextual para marcarlo como clave primaria.
3. Comprobar visualmente que el atributo queda representado como identificador.



Figura E.4: Entidades *Empleados* y *Oficinas* con sus atributos y claves primarias.

Este primer paso permite construir la parte básica del modelo antes de incorporar relaciones entre entidades.

Crear una relación entre entidades

Una vez creadas las entidades *Empleados* y *Oficinas*, se añade una relación entre ambas. En el ejemplo se utiliza la relación *Trabaja_en*, que representa que los empleados trabajan en oficinas.

Para crear la relación:

1. Arrastrar una relación desde el panel lateral hasta el lienzo.
2. Cambiar el nombre de la relación a *Trabaja_en*.
3. Seleccionar la relación creada.
4. Abrir la acción contextual de configuración de participantes.
5. Seleccionar como participantes las entidades *Empleados* y *Oficinas*.
6. Confirmar la configuración.



Figura E.5: Configuración de los participantes de la relación *Trabaja_en*.

La relación todavía no está completa hasta que se indiquen sus cardinalidades. En este ejemplo se quiere representar que una oficina puede tener cero o varios empleados y que cada empleado trabaja en una única oficina. Para ello, se configura la relación con cardinalidad $(0,n)$ asociada a *Empleados* y $(1,1)$ asociada a *Oficinas*, de acuerdo con la notación utilizada en el editor.

Para configurar las cardinalidades:

1. Seleccionar la relación *Trabaja_en*.

2. Abrir la acción contextual de cardinalidades.
3. Definir la cardinalidad correspondiente a cada participante.
4. Confirmar los cambios.
5. Revisar visualmente que las cardinalidades aparecen en el diagrama.

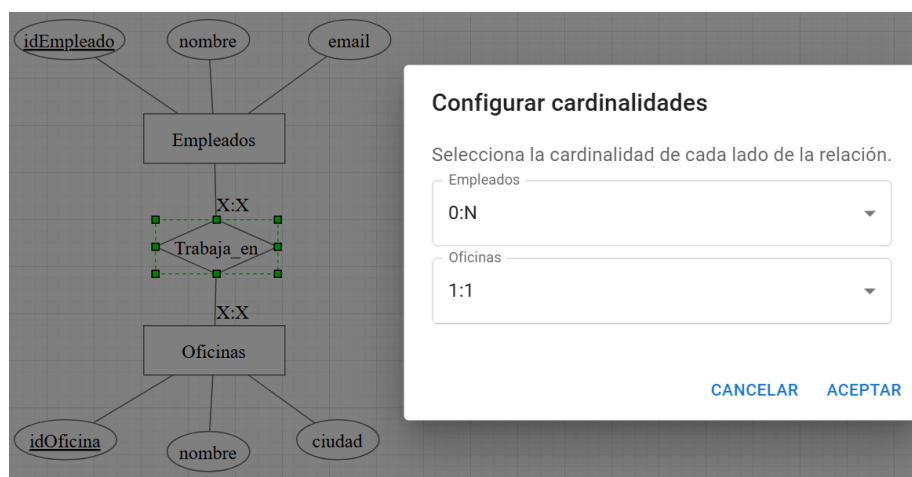


Figura E.6: Configuración de cardinalidades de la relación *Trabaja_en*.

La correcta configuración de participantes y cardinalidades es necesaria para que la aplicación pueda validar el diagrama y transformar posteriormente el modelo E/R al modelo relacional.

Validar el diagrama

Antes de generar el SQL, es recomendable comprobar que el diagrama está completo y no contiene errores básicos. La validación revisa aspectos como la existencia de claves primarias, la configuración de relaciones, la ausencia de nombres repetidos y la definición correcta de cardinalidades.

Para validar el diagrama:

1. Revisar que las entidades principales tienen atributos.
2. Comprobar que cada entidad tiene definida su clave primaria.
3. Confirmar que las relaciones tienen participantes y cardinalidades.

4. Pulsar la acción *Comprobar diagrama* en el panel lateral.
5. Leer el diagnóstico mostrado por la aplicación.

Si el diagrama contiene algún problema, la aplicación muestra un mensaje de validación para indicar qué debe corregirse. Por ejemplo, si la entidad *Empleados* no tuviera atributos o no tuviera clave primaria, el diagnóstico debería señalar ese problema sobre la entidad concreta del ejemplo.

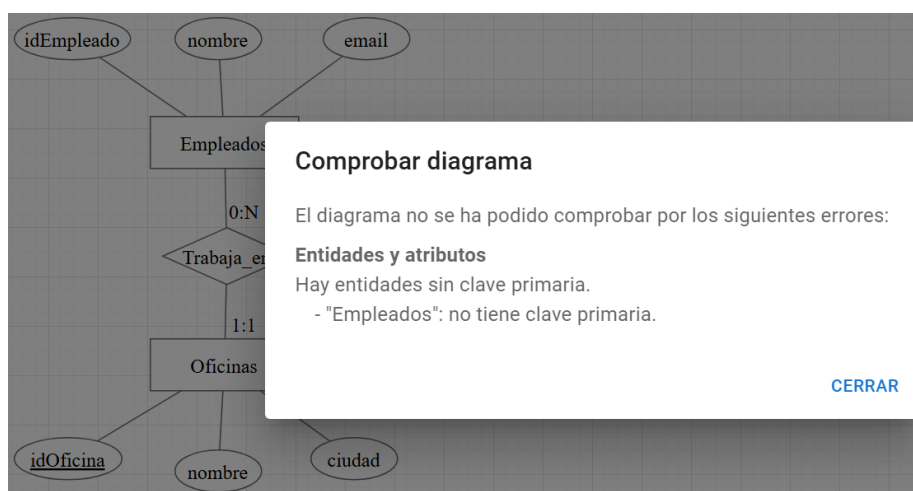


Figura E.7: Mensaje de validación asociado a la entidad *Empleados*.

En la figura se muestra un ejemplo de diagnóstico cuando la entidad *Empleados* no tiene definida una clave primaria. El mensaje identifica el elemento concreto que debe revisarse, lo que facilita corregir el diagrama antes de generar el SQL.

Generar SQL

Cuando el diagrama está validado, la aplicación permite generar un script SQL a partir del modelo E/R construido. En el ejemplo de empleados y oficinas, la generación produce una representación relacional con las tablas, claves primarias y claves ajenas necesarias para reflejar la relación entre ambas entidades.

Para generar el SQL:

1. Comprobar previamente el diagrama.

2. Corregir los errores indicados por la validación, si los hubiera.
3. Pulsar la acción *Generar SQL*.
4. Revisar el script mostrado por la aplicación.
5. Copiar o guardar el resultado si se desea utilizar como referencia.

```
DROP TABLE IF EXISTS Oficinas, Empleados CASCADE;

CREATE TABLE Oficinas (
    idOficina VARCHAR(40) PRIMARY KEY,
    nombre VARCHAR(40),
    ciudad VARCHAR(40)
);

CREATE TABLE Empleados (
    idEmpleado VARCHAR(40) PRIMARY KEY,
    nombre VARCHAR(40),
    email VARCHAR(40),
    idOficina_Trabaja_en VARCHAR(40) NOT NULL REFERENCES Oficinas
);
```

Figura E.8: Script SQL compatible con PostgreSQL generado a partir del ejemplo.

El SQL generado por la aplicación está orientado a PostgreSQL. La salida incluye sentencias de creación de tablas, claves primarias y claves ajenas derivadas de la transformación relacional del diagrama. No obstante, debe interpretarse como una ayuda académica para comprender la correspondencia entre el modelo E/R y el modelo relacional, no como un generador industrial completo preparado para cubrir todas las decisiones físicas de una base de datos real.

Guardar, importar y exportar diagramas

La aplicación permite guardar y recuperar diagramas mediante ficheros JSON. Esta funcionalidad resulta útil para conservar el trabajo realizado, compartir un diagrama con otra persona o continuar editándolo en otro momento.

Para guardar el diagrama actual:

1. Pulsar la acción de exportación JSON.

2. Guardar el fichero generado en el equipo.
3. Conservar ese fichero si se desea reabrir posteriormente el diagrama en la aplicación.

El fichero JSON contiene la información necesaria para reconstruir el diagrama, incluyendo sus elementos, configuración y posiciones en el lienzo.

Para importar un diagrama:

1. Pulsar la acción de importación JSON.
2. Seleccionar el fichero que se desea cargar.
3. Elegir si el contenido importado debe reemplazar el diagrama actual o combinarse con él.
4. Confirmar la operación.



Figura E.9: Importación de un diagrama desde un fichero JSON.

La opción de reemplazar resulta adecuada cuando se desea abrir un diagrama guardado y trabajar únicamente sobre él. En cambio, la opción de combinar permite añadir los elementos importados al diagrama que ya está abierto.

Además de la exportación JSON, la aplicación permite exportar una representación visual del diagrama. Esta opción genera una imagen del lienzo en formatos como PNG o SVG, pensada para incluir el diagrama en

documentos, entregas o material de apoyo. A diferencia del JSON, estos formatos visuales no están pensados para volver a editar el diagrama dentro de la aplicación, sino para obtener una copia gráfica del resultado.

Acciones de edición y visualización

Durante la construcción del diagrama, el usuario puede modificar los elementos creados directamente desde el lienzo. Las acciones disponibles dependen del tipo de elemento seleccionado.

Al seleccionar una entidad, pueden aparecer acciones para añadir atributos, mostrar u ocultar atributos asociados, marcar la entidad como débil, cambiar su orden visual o eliminarla. Al seleccionar un atributo, pueden aparecer acciones para marcarlo como clave primaria, convertirlo en multivaluado, crear subatributos o eliminarlo. Al seleccionar una relación, pueden configurarse sus participantes, cardinalidades y atributos propios cuando proceda.

La aplicación también permite seleccionar varios elementos a la vez. Esta funcionalidad resulta útil cuando el usuario quiere reorganizar una parte del diagrama o eliminar varios elementos en una única operación.

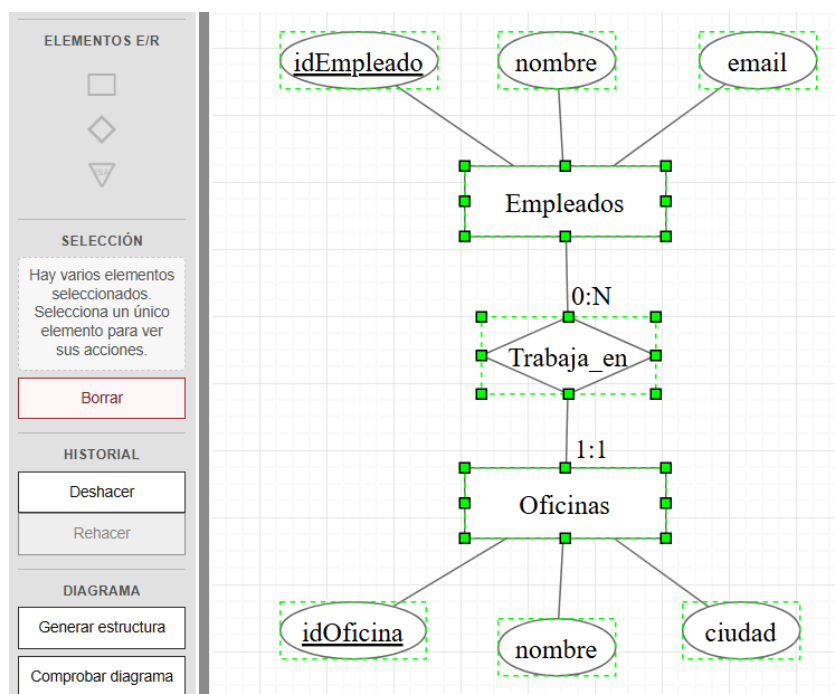


Figura E.10: Selección múltiple de elementos en el lienzo.

Cuando existe una selección múltiple, la interfaz evita mostrar acciones individuales que solo tendrían sentido para un tipo concreto de elemento. En su lugar, se mantienen disponibles las operaciones aplicables al conjunto seleccionado, como mover o eliminar los elementos seleccionados.

Entre las acciones generales de visualización también se encuentra el ajuste de vista, que centra y adapta el contenido del lienzo para facilitar la revisión del diagrama completo. Esta opción resulta útil cuando el modelo ha crecido o cuando algunos elementos han quedado alejados de la zona visible.

Generar estructuras predefinidas

Además de crear los elementos de forma manual, la aplicación permite generar estructuras predefinidas de ejemplo. Esta funcionalidad sirve como apoyo para comenzar rápidamente un diagrama sencillo, probar el comportamiento del editor o revisar cómo se representan determinadas combinaciones de elementos.

Para generar una estructura predefinida:

1. Seleccionar la acción de generación de estructuras en el panel lateral.
2. Elegir el tipo de estructura que se desea insertar.
3. Indicar si la estructura debe reemplazar el diagrama actual o combinarse con el contenido existente.
4. Confirmar la operación.

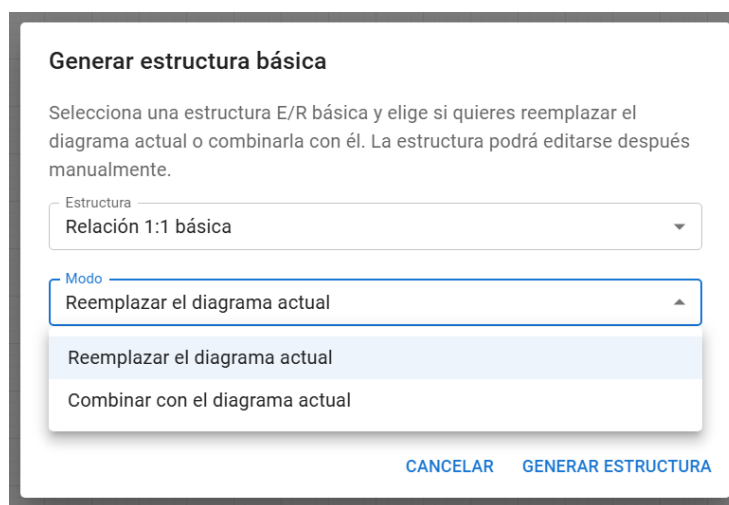


Figura E.11: Generación de estructuras predefinidas en el editor.

La opción de reemplazar resulta adecuada cuando se quiere empezar desde una estructura limpia. La opción de combinar permite añadir la estructura generada al diagrama que ya se está editando. En este segundo caso, el usuario debe revisar visualmente el resultado para comprobar que la combinación mantiene el sentido del modelo.

Esta funcionalidad no sustituye al modelado manual del diagrama, sino que actúa como una ayuda para crear ejemplos, realizar pruebas o acelerar la construcción inicial de un modelo sencillo.

Uso de elementos avanzados

UBU E-R App incorpora soporte para algunos elementos avanzados del modelo Entidad-Relación dentro del alcance definido en este TFG. Estos elementos permiten construir diagramas más completos, aunque requieren una configuración cuidadosa para que la validación y la generación SQL sean coherentes.

Entidades débiles

Para utilizar una entidad débil, el usuario debe crear primero la entidad propietaria y la entidad que dependerá de ella. Después debe marcar la entidad correspondiente como débil, configurar la relación identificadora y definir el discriminante.

El flujo habitual es el siguiente:

1. Crear la entidad propietaria.
2. Crear la entidad que se desea tratar como entidad débil.
3. Marcar la entidad correspondiente como débil desde sus acciones contextuales.
4. Configurar la relación identificadora con la entidad propietaria.
5. Definir el discriminante de la entidad débil.
6. Validar el diagrama.

Si falta la entidad propietaria, la relación identificadora o el discriminante, la validación informa del problema para que el usuario pueda corregirlo.

Relaciones ternarias y roles

Las relaciones ternarias permiten representar interrelaciones en las que participan tres entidades. Para utilizarlas, el usuario debe crear la relación, configurar sus participantes y definir sus cardinalidades.

El flujo habitual es el siguiente:

1. Crear las entidades participantes.
2. Crear la relación en el lienzo.
3. Abrir la configuración de la relación desde las acciones contextuales.
4. Seleccionar los tres participantes.
5. Definir las cardinalidades correspondientes.
6. Añadir roles cuando una entidad participe más de una vez o cuando sea necesario aclarar el significado de cada participación.

Los roles ayudan a distinguir participaciones ambiguas, especialmente cuando una misma entidad interviene en más de un papel dentro de la relación.

Jerarquías ISA

La aplicación permite representar jerarquías ISA sencillas, formadas por una entidad general y una o varias especializaciones. Este soporte es inicial y controlado, por lo que no incluye discriminantes ISA, herencia múltiple real ni restricciones de totalidad, parcialidad, solapamiento o exclusividad.

El flujo habitual es el siguiente:

1. Crear la entidad que actuará como generalización.
2. Crear una o varias entidades especializadas.
3. Insertar el elemento ISA en el lienzo.
4. Configurar la entidad general y sus especializaciones mediante las acciones disponibles.
5. Validar el diagrama para detectar configuraciones incompletas o no soportadas.

En la transformación relacional soportada por la aplicación, la clave de la entidad general se hereda en las especializaciones. Por ello, las entidades especializadas no deben definir una clave propia independiente.

Ayuda, idioma e información del proyecto

La aplicación incluye varias acciones de apoyo pensadas para facilitar su uso y proporcionar información adicional al usuario.

El selector de idioma permite alternar entre español e inglés. Al cambiar el idioma, la interfaz actualiza sus textos principales, incluyendo botones, mensajes de ayuda y diagnósticos mostrados al usuario.

La acción de ayuda muestra una guía breve sobre el funcionamiento general de la aplicación. Esta guía sirve como referencia rápida para recordar cómo crear elementos, utilizar las acciones contextuales, validar el diagrama o generar SQL.

La opción de información del proyecto muestra datos generales sobre la aplicación, incluyendo su nombre, autoría, créditos, tecnologías utilizadas, referencia al proyecto original y licencia de la versión actual.

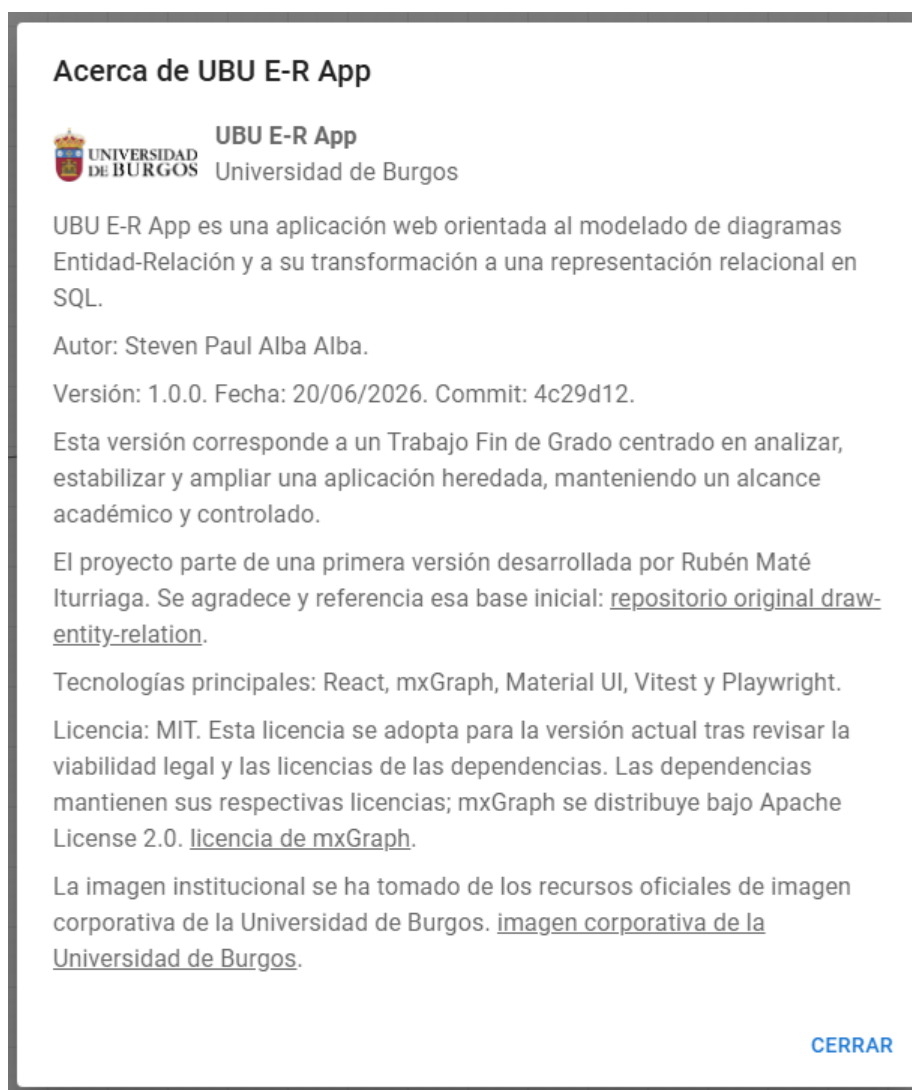


Figura E.12: Información general de UBU E-R App.

Estas acciones complementan el flujo principal de edición y contribuyen a que la aplicación sea más comprensible para usuarios que acceden por primera vez al editor.

Flujo de trabajo recomendado

Aunque la aplicación permite editar el diagrama de forma flexible, se recomienda seguir un flujo de trabajo ordenado para reducir errores y facilitar la generación posterior del SQL.

De forma resumida, el flujo habitual de uso es el siguiente:

1. Crear las entidades principales del modelo.
2. Añadir atributos y definir claves primarias.
3. Incorporar relaciones entre entidades.
4. Configurar participantes y cardinalidades.
5. Añadir elementos adicionales, como entidades débiles, atributos de relación, relaciones ternarias o estructuras ISA, cuando proceda.
6. Comprobar el diagrama mediante la validación.
7. Generar y revisar el SQL compatible con PostgreSQL.
8. Exportar el diagrama en JSON si se quiere conservar para futuras ediciones.
9. Exportar el diagrama como imagen si se quiere incluir en documentación o material de apoyo.

Este flujo no es obligatorio, pero ayuda a mantener el diagrama coherente y facilita que la transformación relacional refleje correctamente el modelo diseñado.

Apéndice F

Anexo de sostenibilización curricular

F.1. Introducción

Este anexo incluye una reflexión personal sobre los aspectos de sostenibilidad abordados durante el desarrollo del Trabajo Fin de Grado. En este contexto, la sostenibilidad no se entiende únicamente desde una perspectiva medioambiental, sino también desde dimensiones educativas, técnicas, sociales y profesionales.

El proyecto desarrollado consiste en la evolución de una aplicación web para la creación, validación y transformación de diagramas Entidad-Relación. Por ello, su relación con la sostenibilidad se sitúa principalmente en el ámbito de la educación, la reutilización de software, la documentación, el mantenimiento responsable y el acceso a herramientas digitales de apoyo al aprendizaje.

A continuación, se relacionan algunas de las competencias trabajadas durante el proyecto con distintos Objetivos de Desarrollo Sostenible de la Agenda 2030 de Naciones Unidas [1].

F.2. Competencias de sostenibilidad adquiridas

ODS 4: Educación de calidad

La aplicación desarrollada puede servir como apoyo para estudiantes que estén aprendiendo el modelo Entidad-Relación y su transformación al modelo relacional. El hecho de poder crear diagramas, validarlos y observar una posible generación de SQL facilita la práctica con ejemplos vistos en clase y permite detectar errores de forma más visual.

Además, al tratarse de una aplicación web, su uso no requiere instalar herramientas complejas. Esto reduce la barrera de entrada para el estudiante y permite centrarse en el aprendizaje del modelo. No obstante, la herramienta debe entenderse como un complemento al estudio y a la explicación del profesorado, no como un sustituto del razonamiento teórico necesario para diseñar correctamente una base de datos.

ODS 9: Industria, innovación e infraestructura

El proyecto se ha basado en la evolución de una herramienta software existente, incorporando nuevas funcionalidades y estabilizando su comportamiento. Esta forma de trabajo se aproxima a situaciones habituales en el ámbito profesional, donde no siempre se parte de cero, sino que es necesario comprender, mantener y ampliar sistemas ya desarrollados.

Durante el trabajo se ha seguido una organización progresiva mediante issues, hitos, pruebas y documentación. Esta metodología ha permitido abordar los cambios de forma controlada, reduciendo el riesgo de introducir modificaciones incoherentes con el resto del sistema. Desde este punto de vista, el proyecto contribuye al desarrollo de una infraestructura digital académica más mantenible y preparada para futuras ampliaciones.

ODS 12: Producción y consumo responsables

Uno de los aprendizajes más importantes del proyecto ha sido comprender que reutilizar software existente también implica responsabilidad. Aunque partir de una aplicación heredada puede parecer más sencillo que comenzar desde cero, en la práctica fue necesario dedicar una parte importante del trabajo a entender su estructura interna, sus decisiones de diseño y las dependencias entre el lienzo, el modelo interno, las validaciones, la transformación relacional y la generación de SQL.

Esta experiencia ha permitido valorar la reutilización como una práctica sostenible, siempre que vaya acompañada de análisis, documentación y mantenimiento. En lugar de descartar el trabajo previo, se ha aprovechado la base existente y se han introducido mejoras de forma progresiva. También se ha prestado atención a la licencia del proyecto actual y a las dependencias utilizadas, con el objetivo de respetar el trabajo de terceros y aclarar las condiciones de uso de la versión desarrollada.

ODS 17: Alianzas para lograr los objetivos

El ODS 17 se centra en la cooperación entre distintos actores para facilitar el desarrollo sostenible. Aunque este objetivo suele asociarse a alianzas institucionales o internacionales, también incluye aspectos relacionados con la transferencia de conocimiento, el acceso a la tecnología y la colaboración entre personas u organizaciones.

En este proyecto, la relación con el ODS 17 se plantea de forma moderada. El trabajo se apoya en software libre y en herramientas abiertas o ampliamente accesibles, como React, mxGraph, Git, GitHub, LaTeX y distintas utilidades de desarrollo y validación. Esta base tecnológica permite reutilizar conocimiento existente, reducir la dependencia de soluciones cerradas y facilitar que otros estudiantes puedan continuar el proyecto sin partir desde cero.

Además, el desarrollo ha requerido una comunicación continua con el tutor académico y una revisión progresiva de las decisiones tomadas. Esta colaboración ha sido especialmente importante porque el modelo Entidad-Relación puede admitir distintos matices de interpretación, y la aplicación debía mantenerse alineada con los materiales docentes y con el criterio seguido en la asignatura.

También se ha intentado dejar una base documentada para que futuros estudiantes o desarrolladores puedan comprender el estado del proyecto, sus funcionalidades, sus limitaciones y sus posibles líneas de continuación. En este sentido, la documentación forma parte de la sostenibilidad del trabajo, ya que facilita que el conocimiento generado no quede limitado únicamente al periodo de desarrollo del TFG.

F.3. Reflexión final

La realización de este Trabajo Fin de Grado me ha permitido trabajar competencias relacionadas con la sostenibilidad desde una perspectiva principalmente educativa y técnica. La contribución más directa del proyecto se relaciona con el ODS 4, ya que la aplicación puede servir como apoyo al aprendizaje del modelo Entidad-Relación, la validación de diagramas y la comprensión de su transformación al modelo relacional y a SQL.

También se han trabajado competencias vinculadas con el ODS 9 y el ODS 12. El proyecto no se ha desarrollado desde cero, sino a partir de una aplicación heredada que ha sido analizada, estabilizada y ampliada de forma incremental. Esta forma de trabajo refuerza la importancia de mantener

software existente, reutilizar recursos técnicos, documentar las decisiones adoptadas y evitar reimplementaciones innecesarias cuando ya existe una base aprovechable.

En relación con el ODS 17, la contribución se entiende de forma prudente. El uso de software libre, la documentación del estado final del proyecto y la colaboración con el tutor académico favorecen la transferencia de conocimiento y facilitan que futuros estudiantes puedan continuar la evolución de la herramienta. No se trata de una alianza institucional amplia, sino de una contribución limitada al contexto académico del proyecto.

También es necesario reconocer algunas limitaciones. El uso de servicios en la nube para el despliegue y la validación de la aplicación puede implicar un consumo adicional de recursos frente a una ejecución exclusivamente local. Del mismo modo, una herramienta de apoyo como esta no debe sustituir el aprendizaje manual del diseño Entidad-Relación, ya que existe el riesgo de que el estudiante dependa de la aplicación sin comprender suficientemente los criterios teóricos. Por ello, la herramienta debe utilizarse como complemento docente y no como reemplazo del razonamiento necesario para diseñar correctamente una base de datos.

En conjunto, considero que el proyecto contribuye de forma modesta pero realista a la sostenibilidad educativa y técnica. Su principal aportación no está en resolver un problema medioambiental directo, sino en mejorar una herramienta académica existente, reducir barreras de uso, documentar su funcionamiento y dejar una base más mantenible para futuras ampliaciones.

Bibliografía

- [1] Naciones Unidas. Objetivos de Desarrollo Sostenible. <https://sdgs.un.org/goals>. [Internet; consultado 21-junio-2026].
- [2] Open Source Initiative. The MIT License. <https://opensource.org/license/mit>. [Internet; consultado 21-junio-2026].
- [3] The Apache Software Foundation. Apache License, Version 2.0. <https://www.apache.org/licenses/LICENSE-2.0.txt>. [Internet; consultado 21-junio-2026].